Pipes in the Desert
Skeleton Planning

E5 - TEAM A

Supervisor:

Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

March 21st, 2026

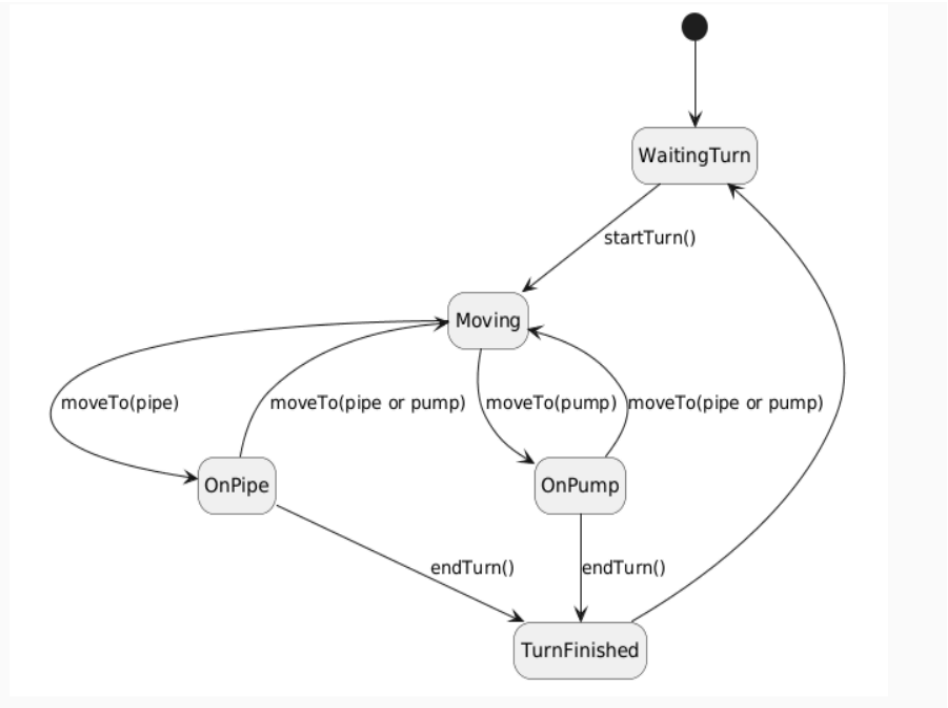
5. Planning the skeleton

5.0 Corrections

5.0.1 State-charts

Player Class:

- **WaitingTurn**: The player is inactive and waiting for their turn.
- **Moving**: The player is moving to an adjacent pipe or pump.
- **OnPipe**: Player is standing on a pipe.
- **OnPump**: Player is standing on a pump.
- **TurnFinished**: The player has finished their turn and becomes inactive again.



5.0.2 AI Consistency Check Corrections

5.0.2.1 (4.7.6 in the document: Analysis Model Final) Sequence Diagram Coverage – Correction

Issue
Some required behaviors were missing from the sequence diagrams.

Correction
Missing diagrams (e.g., Puncture Pipe, Repair Pipe/Pump, Change Pump Direction, Insert Pump, Random Pump Break, Generate Pipes/Pumps) were added.

Result
All minimum required behaviors are now covered and consistent with use cases and functional requirements.

5.0.2.2 (4.7.7) Static Model vs Sequence Diagram – Correction

Issue
The Move Player diagram incorrectly modified network structure, and some relationships (e.g., System–WaterFlowManager) were unclear.

Correction
Movement now only validates adjacency and updates position. Structural operations were removed from movement and kept only for network changes. Relationships and naming were clarified.

Result
Static model and sequence diagrams are now behaviorally and structurally consistent.

5.0.2.2 (4.7.8) State Diagram – Correction

Issue
The Player statechart included undefined operations and overlapping responsibilities.

Correction
The statechart was simplified, undefined operations removed, and responsibilities aligned with Player, Plumber, and Saboteur.

Result
State diagrams are now consistent with class definitions and sequence diagrams.

5.0.2.3 (4.7.10) Bidirectional Traceability – Correction

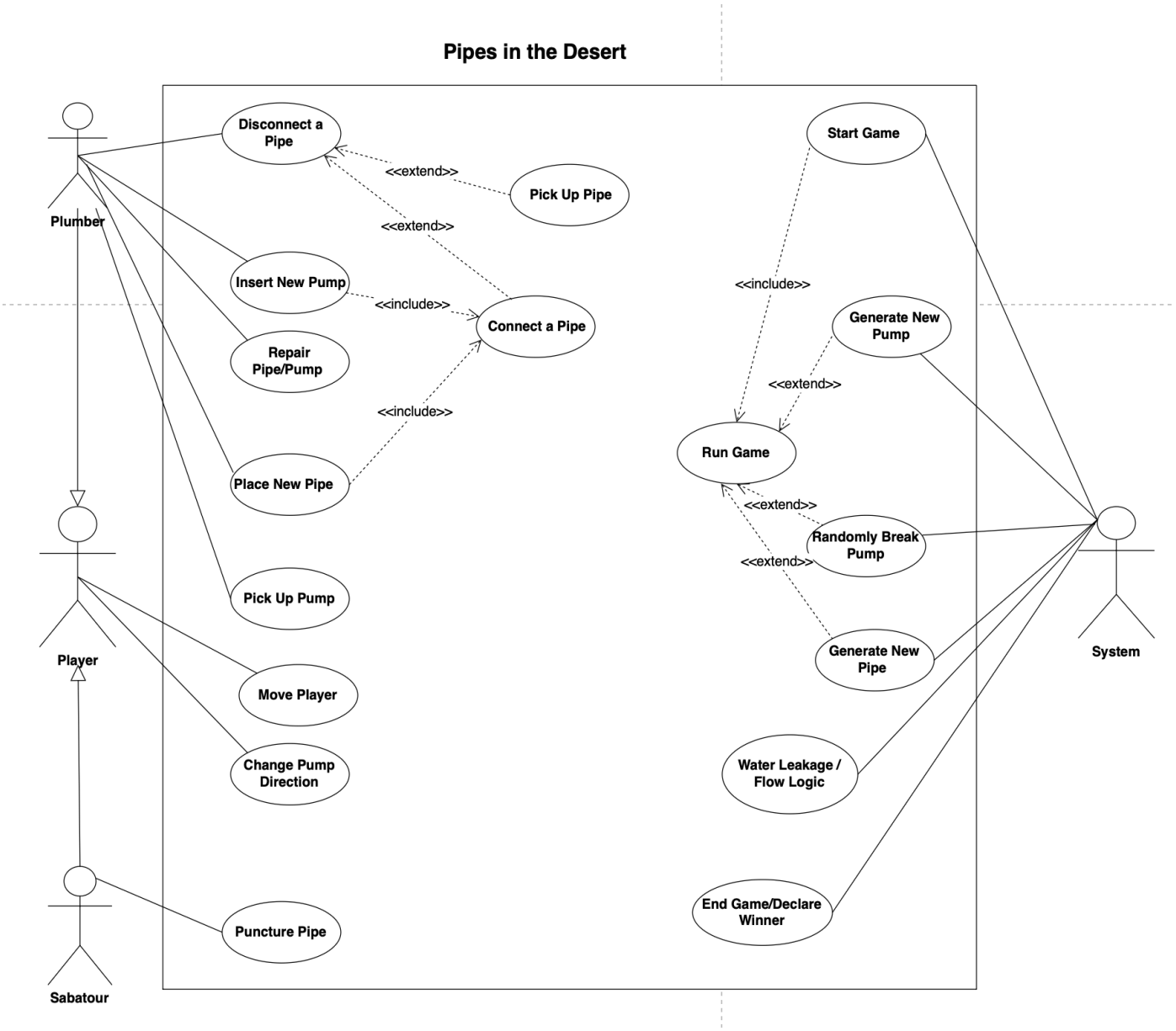
Issue
Traceability was limited due to broad use cases, missing sequence diagrams, indirect concepts, and timer-based interpretation.

Correction
FR references were made explicit, missing diagrams were added, indirect concepts were documented, and the timer-based ending was justified (FR-41).

Result
Traceability is improved and mostly complete, with remaining limitations justified by modeling decisions.

5.1 Real use-cases of the skeleton model

5.1.1 Use-case diagram



5.1.2 Use-case descriptions

Use-case name	**Start Game**
Short textual description	The system initializes a new game session by receiving player and team data, setting the game timer, generating the initial pipe network, and starting the running state of the game.
Actors	System (secondary), Player(s) (primary)
Dialog, scenario	1. System asks for the required initial data: player names, team assignments, and game duration. 2. Player(s) enter the requested data. 3. System validates that the start conditions are satisfied, including valid player setup and timer value. 4. System initializes the game session and creates the required core objects, including, Players and Team, GameTimer, ScoreBoard, PipeNetwork, and WaterFlowManager. 5. System generates the initial network containing at least one spring, one cistern, one pump, and connecting pipes.

Use-case name	**Run Game**
Short textual description	The system supervises an active game session by updating turn order, triggering periodic events, recalculating water flow, and checking whether the game should end.
Actors	System (secondary)
Dialog, scenario	1. The game session is already active. 2. The system updates the current round state. → System.updateRoundEvents() 3. The system recalculates water flow in the network. → WaterFlowManager.recalculateFlow() 4. The system advances control to the next player. → System.nextTurn() 5. The system checks whether the end condition has been reached. → System.checkEndCondition() / GameTimer.hasExpired() 6. If the end condition is reached, the system terminates the session. → System.endGame()

Use-case name	**Move Player**
Short textual description	A player moves to an adjacent connected element that is limited to a pipe or a pump. Springs and cisterns are not valid movement destinations, but they may be interacted with from an adjacent pipe or pump.
Preconditions	● It is the player's turn. ● The game session is active. ● The target element is adjacent to the player's current position.

	● Only pipes and pumps are valid movement destinations.
Actors	Player
Dialog, scenario	1. The player selects a target element. 2. The system checks whether the target is adjacent to the current position. → PipeNetwork.areAdjacent(a, b) 3. The system checks whether the target is a valid destination type (pipe or pump). 4. If the target is a pipe, the system checks whether it is already occupied by another player → Pipe.isOccupied() 5. If the pipe is occupied by another player(if there is already one player on it), move is invalid. 6. If the move is valid, the player's position is updated. → Player.moveTo(target), Player.setPosition(target) 7. If the move is invalid, the system displays an error message.

Use-case name	**Puncture Pipe**
Short textual description	A saboteur punctures an adjacent intact pipe and causes water leakage.
Precondition	● The saboteur is standing on a pipe or pump adjacent to the target pipe. ● The target pipe is intact.
Actors	Saboteur
Dialog, scenario	1. The saboteur selects an adjacent pipe to puncture. 2. The system checks whether the selected pipe is adjacent to the saboteur's current position. 3. The system checks whether the pipe is intact. → Pipe.isPunctured() 4. If valid, the pipe changes to punctured state. → Saboteur.puncturePipe(pipe), Pipe.puncture() 5. The system recalculates water flow. → WaterFlowManager.recalculateFlow() 6. The leaked water is registered as saboteur score. → WaterFlowManager.registerLeakedWater(amount)

Use-case name	**Repair Pipe/Pump**
Short textual description	A plumber repairs a punctured pipe or a broken pump.
Actors	Plumber
Dialog, scenario	1. The plumber selects the damaged element on which they are standing or adjacent to. 2. The system checks whether the selected element is repairable and currently damaged. → Pipe.isPunctured() / Pump.isBroken()

	3. If the target is a pipe, the pipe is repaired. → Plumber.repairPipe(pipe), Pipe.repair() 4. If the target is a pump, the pump is repaired. → Plumber.repairPump(pump), Pump.repair() 5. The system recalculates water flow. → WaterFlowManager.recalculateFlow()

Use-case name	**Change Pump Direction**
Short textual description	A player standing on a working pump changes its active input and output direction.
Actors	Plumber, Saboteur
Dialog, scenario	1. The player selects a pump on which they are standing. 2. The system checks whether the pump is in working state. → Pump.isBroken() 3. The player selects the new active input and output pipes. 4. The pump updates its direction. → Plumber.changePumpDirection(...) / Saboteur.changePumpDirection(...), Pump.setDirection(input, output) 5. The system recalculates water flow. → WaterFlowManager.recalculateFlow() 6. The updated routing is displayed.

Use-case name	**Randomly Break Pump**
Short textual description	During the game session, the system selects a working pump and changes it to broken state.
Actors	System
Dialog, scenario	1. The system performs a round update. → System.updateRoundEvents() 2. The system selects a pump eligible for breakdown. → PipeNetwork.getWorkingPumps() 3. The selected pump changes to broken state. → Pump.breakDown() 4. The system recalculates water flow. → WaterFlowManager.recalculateFlow() 5. The updated pump state is displayed.

Use-case name	**Disconnect a Pipe**
Short textual description	A plumber disconnects one end of an adjacent pipe, creating a free end in the network.
Actors	Plumber
Dialog, scenario	1. The plumber selects an adjacent pipe whose end is to be disconnected. 2. The system checks whether the selected pipe is adjacent to the plumber's current position.

	3. The system checks whether the selected pipe is occupied by another player. → Pipe.isOccupied() 4. If the pipe is occupied by another player, the action is rejected. 5. If valid, the selected pipe disconnects one end. → Pipe.disconnectEnd() 6. The network updates the affected connection. → PipeNetwork.disconnectElements(a, b) 7. One end of the pipe becomes free. 8. The system recalculates the water flow. → WaterFlowManager.recalculateFlow()

Use-case name	**Connect a Pipe**
Short textual description	A plumber connects the free end of an existing pipe to an adjacent network element.
Actors	Plumber
Preconditions	- There exists a pipe with a free end - The plumber is located on a pipe or pump adjacent to the free end - The target element is adjacent and can accept a connection
Postconditions	- The pipe's free end is connected to the selected element - The network structure is updated
Dialog, scenario	1. The plumber selects the pipe with a free end. → Pipe.hasFreeEnd() 2. The plumber selects an adjacent target element. 3. The system checks adjacency and connection validity. → PipeNetwork.areAdjacent(a, b) 4. The pipe connects its free end to the selected target. → Plumber.connectPipeEnd(pipe, target), Pipe.connectFreeEnd(target) 5. The network updates its structure. → PipeNetwork.connectElements(a, b) 6. The system recalculates water flow. → WaterFlowManager.recalculateFlow() 7. The system displays the updated network state

Use-case name	**Pick Up Pump at Cistern**
Short textual description	A plumber standing on an adjacent pipe or pump collects a generated pump from a cistern.
Actors	Plumber
Dialog, scenario	1. The plumber selects a cistern adjacent to the current position. 2. The system checks whether the cistern has an available pump. → Cistern.hasAvailablePump() 3. The plumber will select "Pick Up Pump" option given by the system.

	4. If valid, the cistern removes one available pump. → Cistern.takePump() 5. The plumber receives the pump. → Plumber.collectPumpFromCistern(cistern) 6. The system confirms the pickup.

Use-case name	**Pick Up Pipe from Cistern**
Short textual description	A plumber standing on an adjacent pipe or pump collects a generated pipe from a cistern.
Actors	Plumber
Dialog, scenario	1. The plumber selects a cistern adjacent to the current position. 2. The system checks whether the cistern has an available pipe. → Cistern.hasAvailablePipe() 3. If valid, the cistern removes one available pipe. → Cistern.takePipe() 4. The plumber receives the pipe for later connection. 5. The system confirms the pickup.

Use-case name	**Insert New Pump**
Short textual description	A plumber inserts a carried pump into an existing pipe by splitting the pipe.
Actors	Plumber
Precondition	The plumber has previously picked up a pump from a cistern.
Dialog, scenario	1. The plumber selects a target pipe. 2. The system checks whether the plumber is carrying a pump. 3. The system validates that insertion is possible. → Pump.canAcceptConnection() 4. The network splits the selected pipe → PipeNetwork.insertPump(targetPipe, pump) 5. The new pump is inserted between the two resulting segments. → Plumber.insertPump(targetPipe, pump, network) 6. The system recalculates water flow. → WaterFlowManager.recalculateFlow()

Use-case name	**Water Leakage / Flow Logic**
Short textual description	The system processes water movement through the network and updates delivered and leaked water.
Actors	System
Dialog, scenario	1. The system starts a water-flow update. → WaterFlowManager.recalculateFlow() 2. Water flow is processed from springs through connected pipes and pumps. → WaterFlowManager.processSpring(spring)

	3. If water reaches a cistern, it is counted as delivered. → WaterFlowManager.registerDeliveredWater(amount) 4. If water reaches a punctured pipe or free end, it is counted as leaked. → WaterFlowManager.registerLeakedWater(amount) 5. The scoreboard is updated accordingly. → ScoreBoard.addDeliveredWater(amount), ScoreBoard.addLeakedWater(amount) 6. The system displays the updated flow result.

Use-case name	**End Game / Declare Winner**
Short textual description	The system ends the session and declares the winning team based on the final water result.
Actors	System
Dialog, scenario	1. The system checks whether the timer-based end condition has been reached. → System.checkEndCondition() / GameTimer.hasExpired() 2. If yes, the system stops the game session. → System.endGame(), GameTimer.stop() 3. The system compares delivered water and leaked water. → ScoreBoard.determineWinner() 4. The system determines the winner(if delivered>leaked plumbers win. If leaked > delivered saboteurs win if leaked=delivered it is a draw). 5. The final result is displayed.

Use-case name	**Generate New Pipe**
Short textual description	The system generates a new pipe at a cistern when the pipe-generation condition is satisfied.
Actors	System
Dialog, scenario	1. The system performs a round update. → System.updateRoundEvents() 2. The system checks whether the pipe-generation condition is satisfied. 3. If valid, the system requests component generation. → System.requestComponentGeneration() 4. The cistern generates a new pipe. → Cistern.generatePipe() 5. The number of connected pipes to the cistern increases. 6. The generated pipe remains available until collected by a plumber.

Use-case name	Generate New Pump
Short textual description	The system generates a new pump at a cistern when the pump-generation condition is satisfied.
Actors	System
Dialog, scenario	1. The system performs a round update. → System.updateRoundEvents() 2. The system checks whether the pump-generation condition is satisfied. 3. If valid, the system requests component generation. → System.requestComponentGeneration() 4. The cistern generates a new pump. → Cistern.generatePump() 5. The number of available pumps at the cistern increases. 6. The generated pump remains available until collected by a plumber.

Use-case name	Place New Pipe
Short textual description	A plumber places a newly collected pipe into the network at a valid position.
Actors	Plumber
Dialog, scenario	1. The plumber selects a placement location for the previously collected pipe. 2. The system checks whether the placement is valid. 3. The system inserts the new pipe into the network. 4. The system updates the connections between elements. 5. The system recalculates water flow. 6. The system displays the updated network state.
Preconditions	- The plumber has a pipe collected from a cistern - The target location in the network is valid for insertion
Postconditions	● A new pipe is added to the network ● The pipe is connected to the selected elements

Clarification: Place New Pipe is used for inserting a newly collected pipe into the network, while Connect a Pipe is used for reconnecting the free end of an already existing pipe.

5.2 Plans of the skeleton's UI, dialogs

1. Start / Supervise Game

startGame <team1Name> <team2Name> <durationSeconds> — Initializes the game with two teams, builds the pipe network and starts the game loop handling ticks, flow calculation and turn management.

User Input:

startGame <team1Name> <team2Name> <durationSeconds>

System Output:

initializeGame()
addMember(player)
getMembers()
getSize()
getName()
addElement(element)
connectElements(a, b)
start()

System Output (each game tick):

tick()
getRemainingTime()
hasExpired() -> false
updateRoundEvents()
requestComponentGeneration()
recalculateFlow()
  getOutputPipe()
  registerLeakedWater(amount)
  addDeliveredWater(amount)
nextTurn()
getCurrentPlayer()
checkEndCondition()

2. Move Player

move <targetElementId> — A player moves to an adjacent element. Checks adjacency and occupancy rules differ between pipes and pumps.

User Input:

move <targetElementId>

System Output:

startTurn()
canAct() -> true
getPosition() -> ne
isAdjacentTo(target) -> true
areAdjacent(ne, target) -> true
getNeighbors() -> neighbors
[alt: target is a Pipe]
  isOccupied() -> false
  setOccupant(player)
  setPosition(target)
[alt: target is a Pump]
  (multiple players allowed)
  setPosition(target)

endTurn()

3. Place New Pipe

placePipe <plumberName> <targetElementId> — A plumber places a pipe from their inventory into the network and connects it to a target element.

User Input:

placePipe <plumberName> <targetElementId>

System Output:

hasPipeInInventory() -> true
insertPipe(newPipe, targetElement) -> ok
connectElements(a, b) -> ok
recalculateFlow() -> ok

(if no pipe in inventory -> hasPipeInInventory() -> false, action rejected)

4. Repair Pipe / Pump

repair <plumberName> <targetElementId> — A plumber repairs a damaged pipe or broken pump adjacent to their position and recalculates flow.

User Input:

repair <plumberName> <targetElementId>

System Output [target is a Pipe]:

isPunctured() -> true
repairPipe(pipe)
  repair() -> ok
recalculateFlow() -> ok

System Output [target is a Pump]:

isBroken() -> true
repairPump(pump)
  repair() -> ok
recalculateFlow() -> ok

(if target is not damaged -> isPunctured()/isBroken() -> false, action rejected)

5. Insert New Pump

insertPump <plumberName> <targetElementId> — A plumber inserts a carried pump into the network by splitting an existing pipe into two segments.

User Input:

insertPump <plumberName> <targetElementId>

System Output:

hasPumpInInventory() -> true

canAcceptConnection() -> true

PipeNetwork.insertPump(targetElement, pump)

  removeElement(targetElement) -> ok

  disconnectElements(a, b) -> ok

  connectPumpBetweenSegments(pump) -> ok

  addConnectedPipe(pipe) -> ok

Plumber.insertPump(targetElement, pump, network)

recalculateFlow()

(if no pump in inventory -> hasPumpInInventory() -> false, action rejected)

6. Change Pump Direction

changePumpDir <playerName> <pumpId> <inputPipeId> <outputPipeId> — A player changes the input/output direction of a working pump, affecting water flow.

User Input:

changePumpDir <playerName> <pumpId> <inputPipeId> <outputPipeId>

System Output:

isBroken() -> false

canTransferWater() -> true

[Player is Plumber]

changePumpDirection(pump, input, output)

  setDirection(input, output) -> ok

recalculateFlow()

  registerDeliveredWater(amount)

[Player is Saboteur]

changePumpDirection(pump, input, output)

  setDirection(input, output) -> ok

recalculateFlow()

  registerDeliveredWater(amount)

(if pump is broken -> isBroken() -> true, action rejected)

7. Puncture Pipe

puncturePipe <saboteurName> <pipeId> — A saboteur punctures an adjacent intact pipe, causing water leakage registered as saboteur score.

User Input:

puncturePipe <saboteurName> <pipeId>

System Output:

isPunctured() -> false
puncturePipe(pipe)
  puncture() -> ok
recalculateFlow()
  registerLeakedWater(amount) -> ok

(if already punctured -> isPunctured() -> true, action rejected)

8. Randomly Break Pump

Automatic (game tick) — Each tick the system automatically selects and breaks a working pump.

System Output (automatic — triggered each game tick):

updateRoundEvents()
getWorkingPumps() -> pumps
breakDown() -> ok
recalculateFlow() -> ok

(if no working pump exists -> getWorkingPumps() returns empty, no action)

9. Connect a Pipe

connectPipe <plumberName> <pipeId> <targetElementId> — A plumber connects the free end of an existing pipe to an adjacent network element.

User Input:

connectPipe <plumberName> <pipeId> <targetElementId>

System Output:

hasFreeEnd() -> true
areAdjacent(a, b) -> true

connectPipeEnd(pipe, targetElement)
  connectFreeEnd(targetElement) -> ok
connectElements(a, b) -> ok
recalculateFlow() -> ok
(if no free end -> hasFreeEnd() -> false, action rejected)
(if not adjacent -> areAdjacent(a, b) -> false, action rejected)

10. Disconnect a Pipe

disconnectPipe <plumberName> <pipeId> — A plumber disconnects a pipe end from its connected element and updates the network.

User Input:

disconnectPipe <plumberName> <pipeId>

System Output:

isOccupied() -> false
disconnectEnd() -> ok
disconnectElements(pipe, targetElement) -> ok
recalculateFlow() -> ok
(if pipe is occupied -> isOccupied() -> true, action rejected)

11. Generate New Pipe

Automatic (game tick) — Each tick the system generates a new pipe at a cistern if conditions are met.

System Output (automatic — triggered each game tick):

updateRoundEvents()
requestComponentGeneration()
  generatePipe() -> ok

(if condition not met -> requestComponentGeneration() skipped, no pipe generated)

12. Generate New Pump

Automatic (game tick) — Each tick the system generates a new pump at a cistern if conditions are met.

System Output (automatic — triggered each game tick):

updateRoundEvents()
requestComponentGeneration()
  generatePump() -> ok

(if condition not met -> requestComponentGeneration() skipped, no pump generated)

13. Pick Up Pump at Cistern

pickupPump <plumberName> <cisternId> — A plumber collects an available pump from a cistern into their inventory.

User Input:

pickupPump <plumberName> <cisternId>

System Output:

hasAvailablePump() -> true
takePump() -> ok
collectPumpFromCistern(cistern) -> ok
(if no pump available -> hasAvailablePump() -> false, action rejected)

14. Pick Up Pipe from Cistern

pickupPipe <plumberName> <cisternId> — A plumber collects an available pipe from a cistern into their inventory.

User Input:

pickupPipe <plumberName> <cisternId>

System Output:

hasAvailablePipe() -> true
takePipe() -> ok
collectPipeFromCistern(cistern) -> ok
(if no pipe available -> hasAvailablePipe() -> false, action rejected)

15. Water Leakage / Flow Logic

Automatic (game tick) — Each tick the system processes water flow from springs, registering delivered or leaked water based on pipe state.

System Output (automatic — triggered each game tick):

recalculateFlow()
processSpring(spring)
produceWater() -> amount
canTransferWater() -> true

System Output [outgoing pipe has free capacity]:

isPunctured() -> false
hasFreeEnd() -> false
receiveWater(amount)
registerDeliveredWater(amount)
addDeliveredWater(amount)

System Output [pipe is punctured or has free end]:

isPunctured() -> true
registerLeakedWater(amount)
addLeakedWater(amount)

16. End Game / Declare Winner

Automatic (timer expires) — When the timer expires the game stops, scores are calculated and the winner is declared.

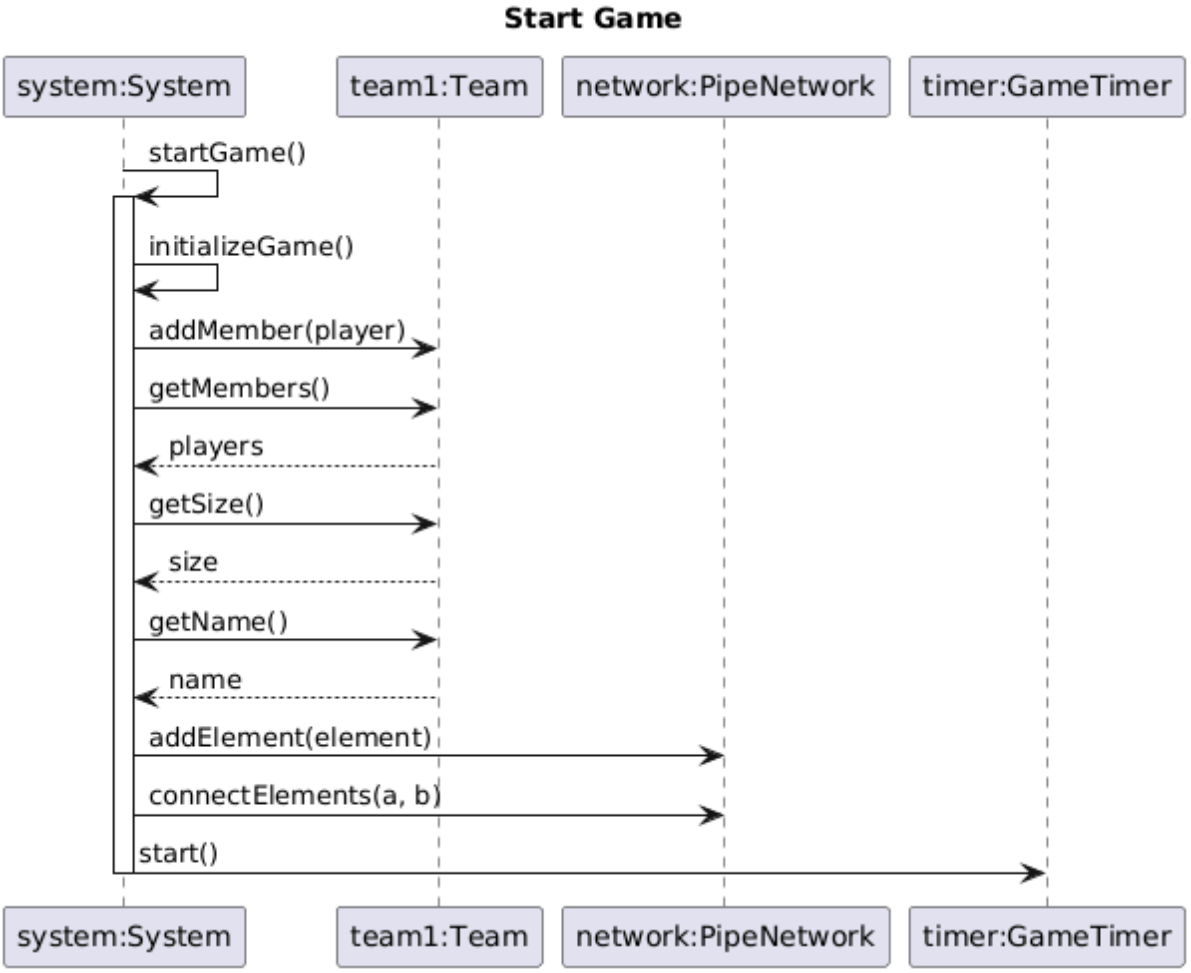
System Output (automatic — when timer expires):

```
tick()
hasExpired() -> true
stop() -> ok
endGame()
getPlumberScore() -> score
getSaboteurScore() -> score
determineWinner() -> winningTeam
```

5.3 Detailed sequence diagrams for internal activities

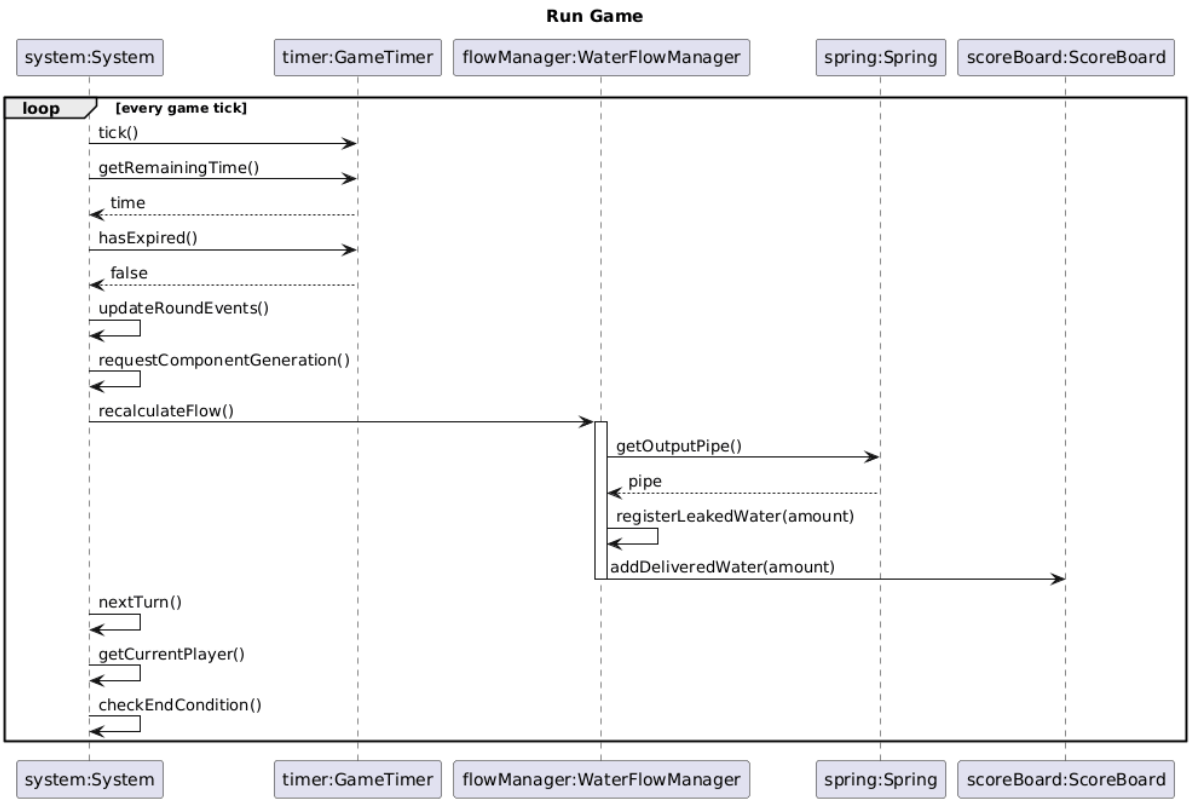
1. **Start Game**

   The System initializes a new game session by setting up teams, generating the initial pipe network with at least one spring, one cistern, one pump and connecting pipes, and starting the game timer. This happens once at the beginning of the session before any player actions are taken.



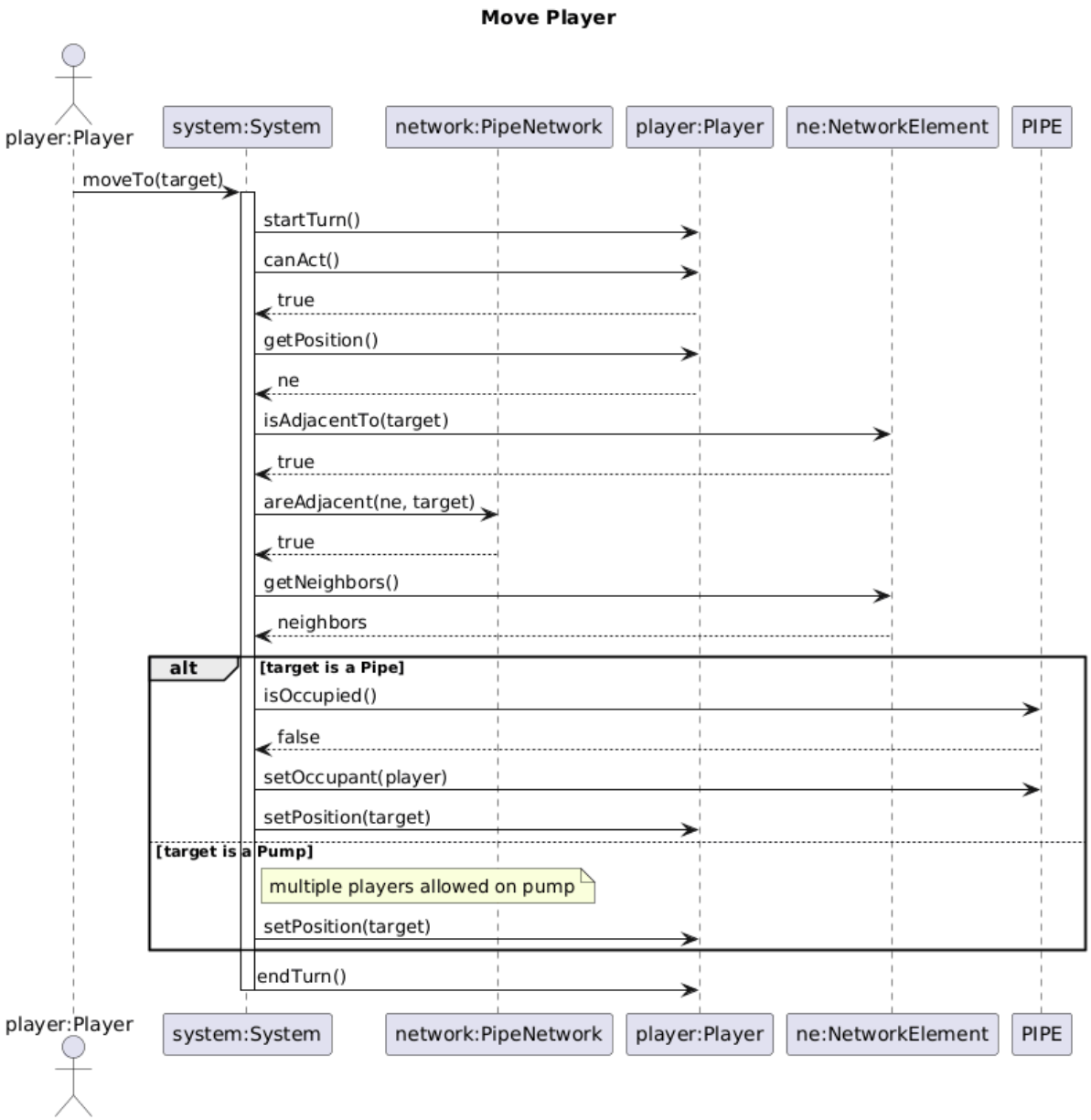
2. **Run Game**

   The System manages the ongoing game session by executing game ticks, handling player turns, triggering round events, coordinating water flow, and updating scores until the timer expires. When the timer expires, the system transitions to End Game.



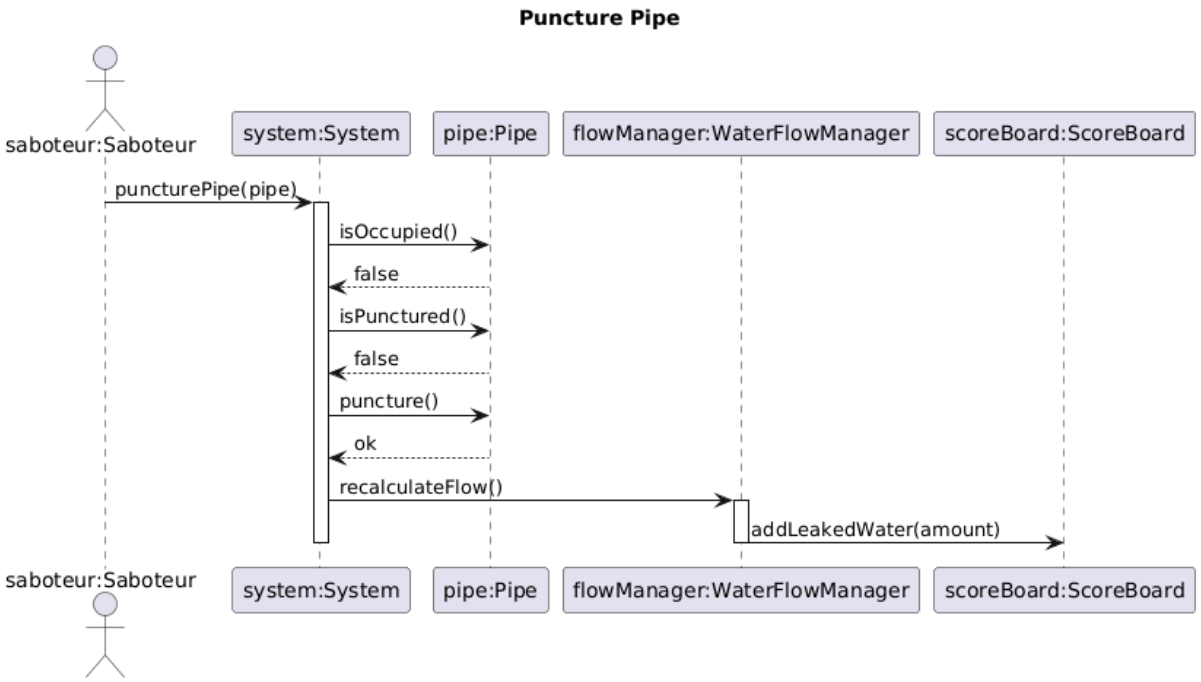
3. **Move Player**

   A player requests to move to an adjacent network element. The System validates the player's current position, checks adjacency, and updates the player's position if the move is valid. According to the problem definition, players can only move onto pipes and pumps, cisterns and springs are not valid destinations. Multiple players may occupy a pump simultaneously, but only one player may occupy a pipe at a time.



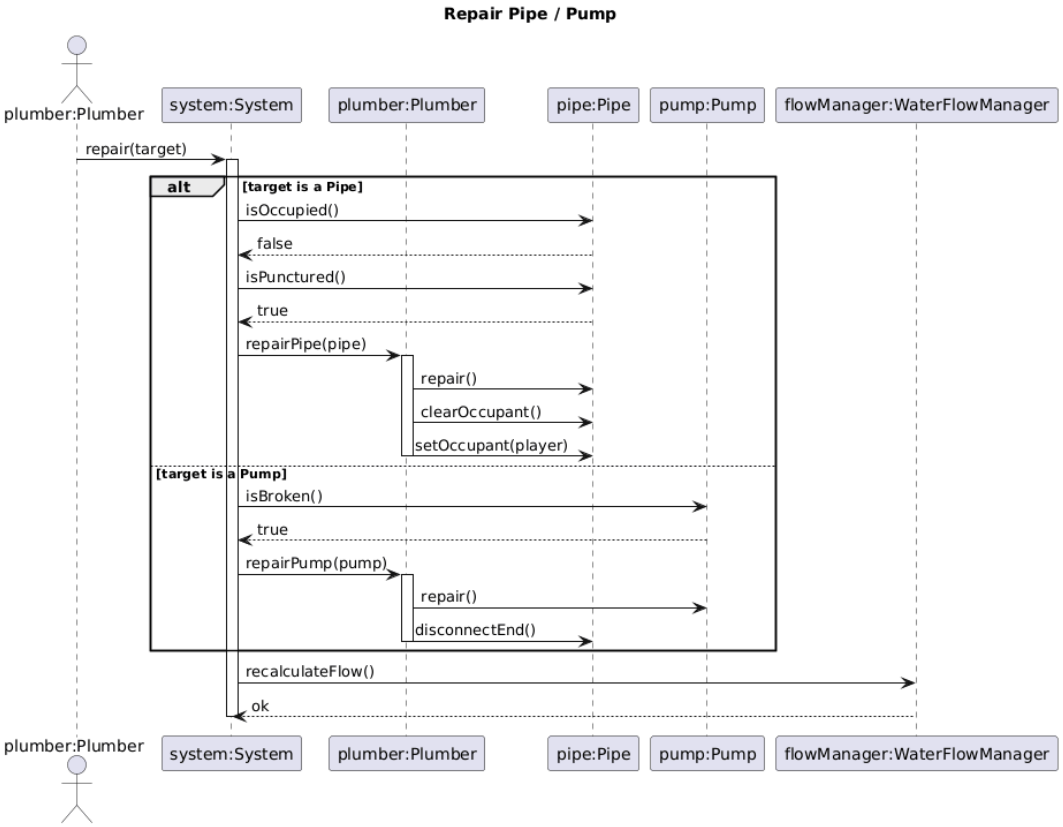
4. **Puncture Pipe**

   A Saboteur standing on a pipe or pump selects an adjacent intact pipe to puncture. The System verifies adjacency and that the target pipe is intact before applying the puncture and updating the score.



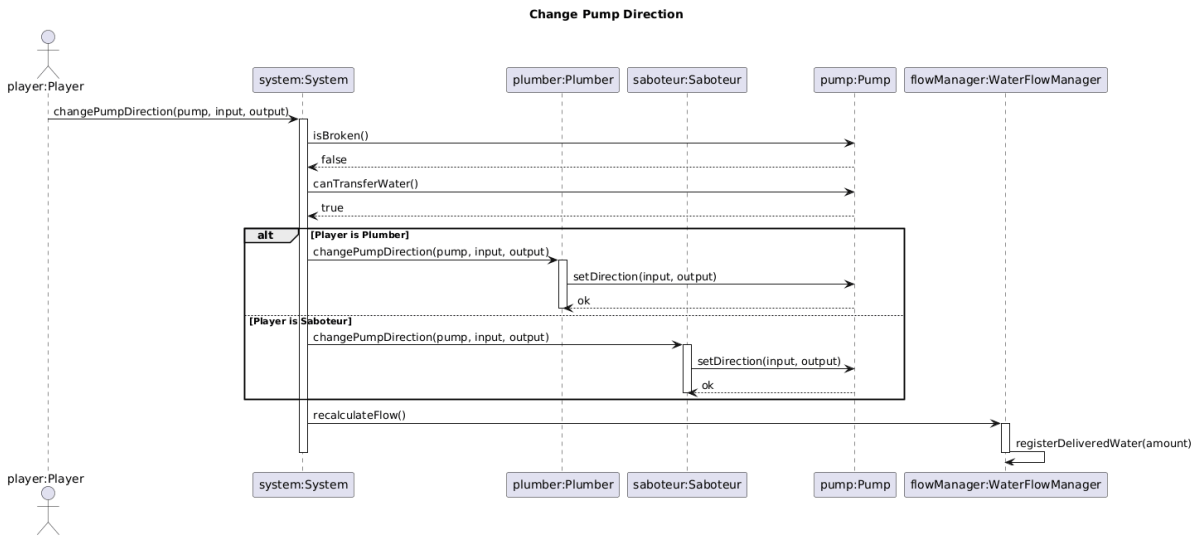
5. **Repair Pipe / Pump**

   A Plumber repairs either a punctured pipe or a broken pump depending on the damaged target element they are standing on or adjacent to. The System validates the state of the target element and restores it to working condition. Water flow is recalculated after the repair is complete.



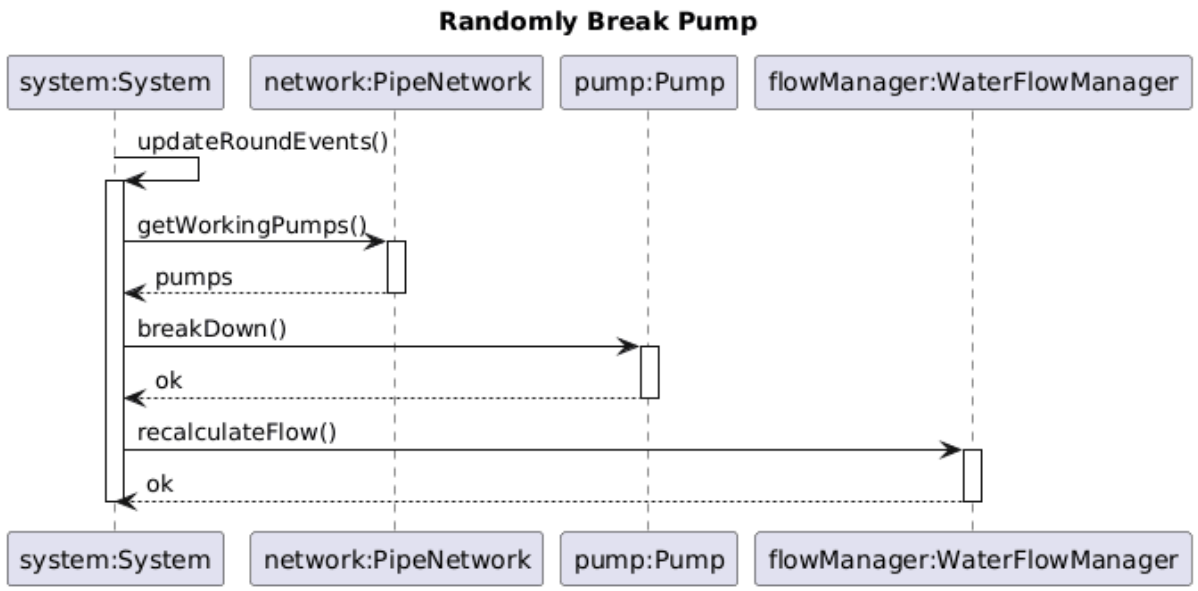
6. **Change Pump Direction**

Either a Plumber or Saboteur standing on a pump changes its active input and output routing. The System first verifies the pump is in working condition before applying the new direction and recalculating water flow across the network.



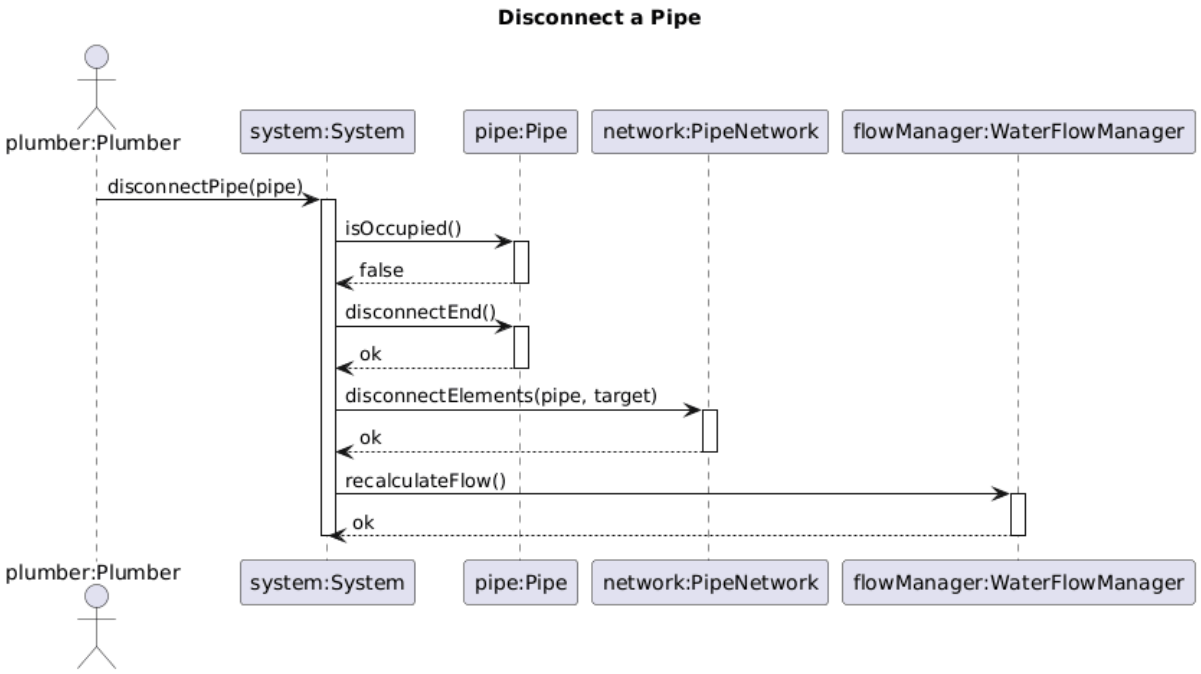
7. **Randomly Break Pump**

The System performs a round update and randomly selects a working pump to break down. The broken pump stops transferring water until a Plumber repairs it. Water flow is recalculated after the breakdown.



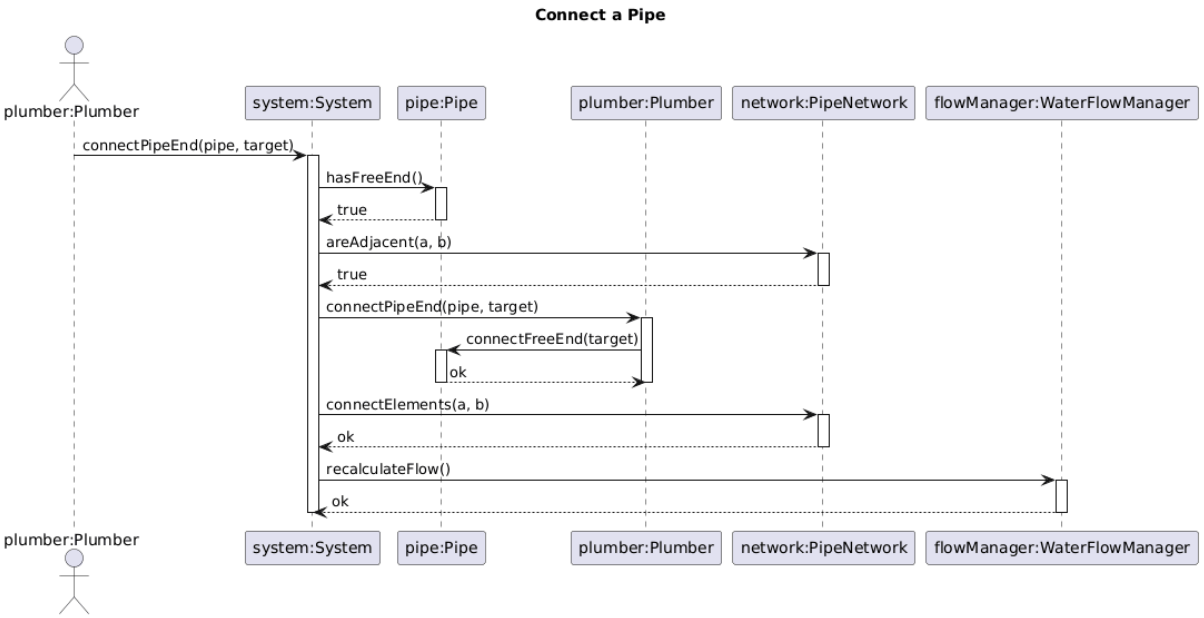
8. **Disconnect a Pipe**

A Plumber standing on a pipe or pump selects an adjacent pipe and disconnects one of its ends from the network. The System verifies the target pipe is not occupied by another player, removes the connection, and recalculates water flow.



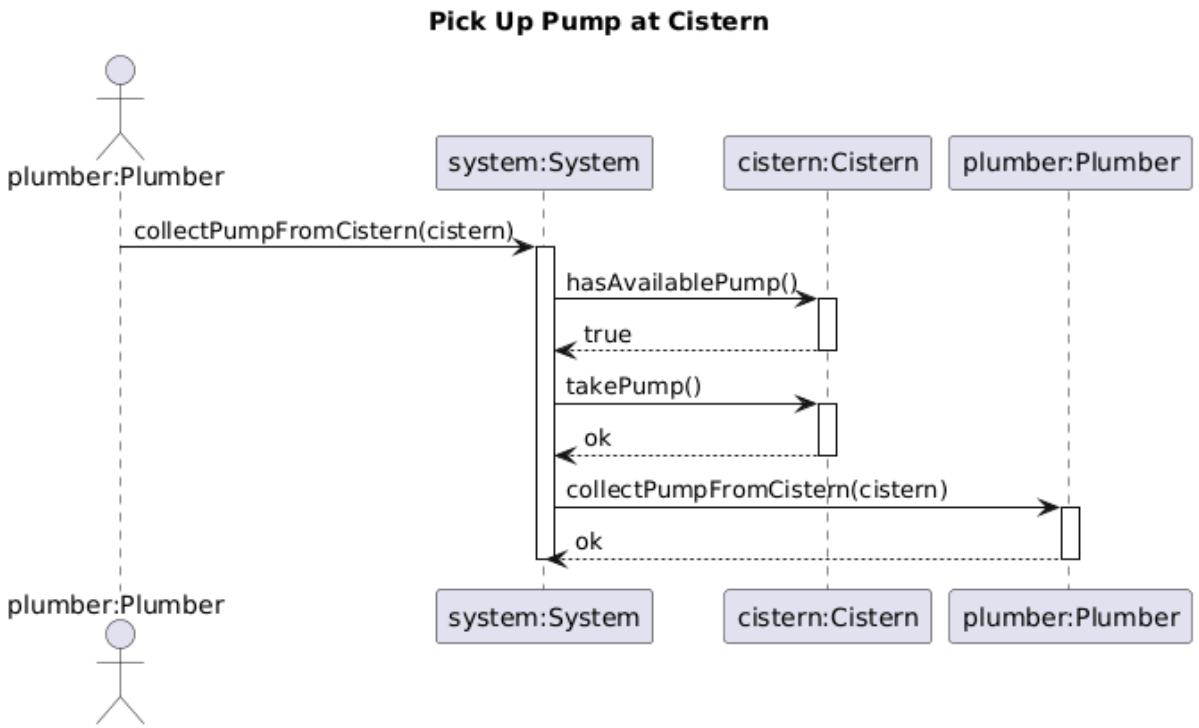
9. **Connect a Pipe**

A Plumber connects the free end of a pipe to an adjacent network element. The System checks adjacency and connection validity before updating the network structure and recalculating water flow.



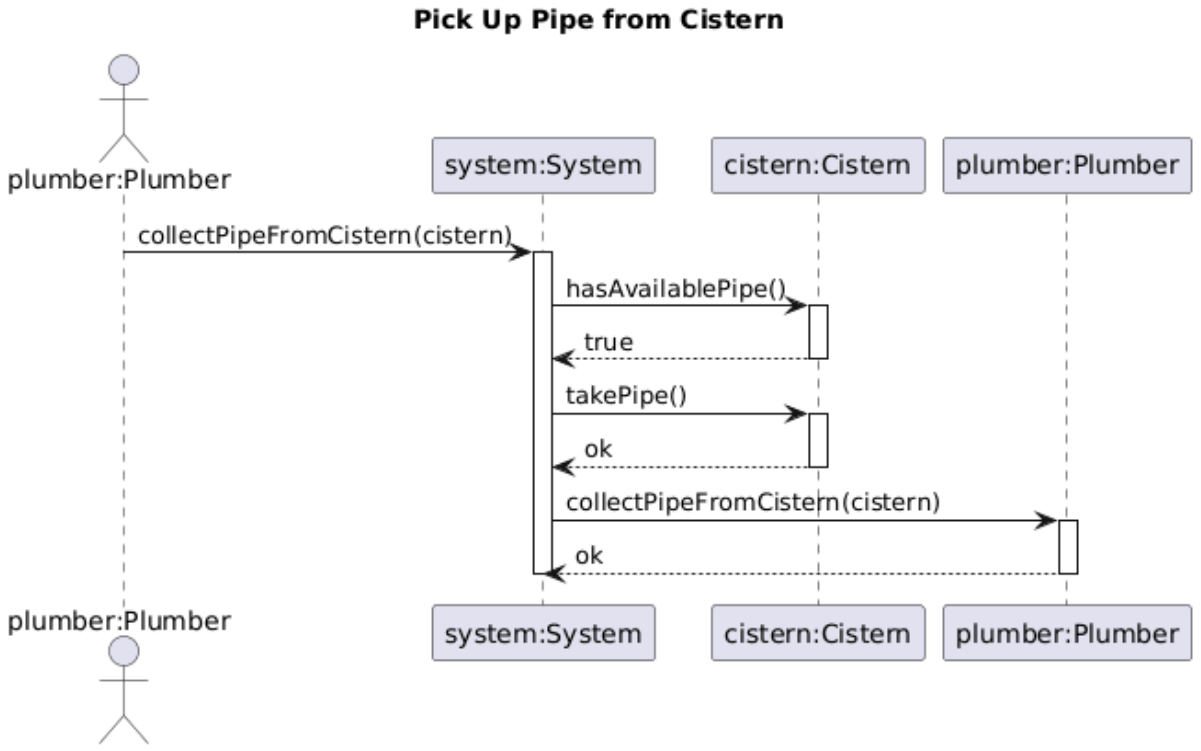
10. **Pick Up Pump at Cistern**

    A Plumber standing on a pipe or pump adjacent to a cistern collects a generated pump from it. The System checks availability and marks the pump as collected. The Plumber can then insert it into a pipe in a subsequent turn.



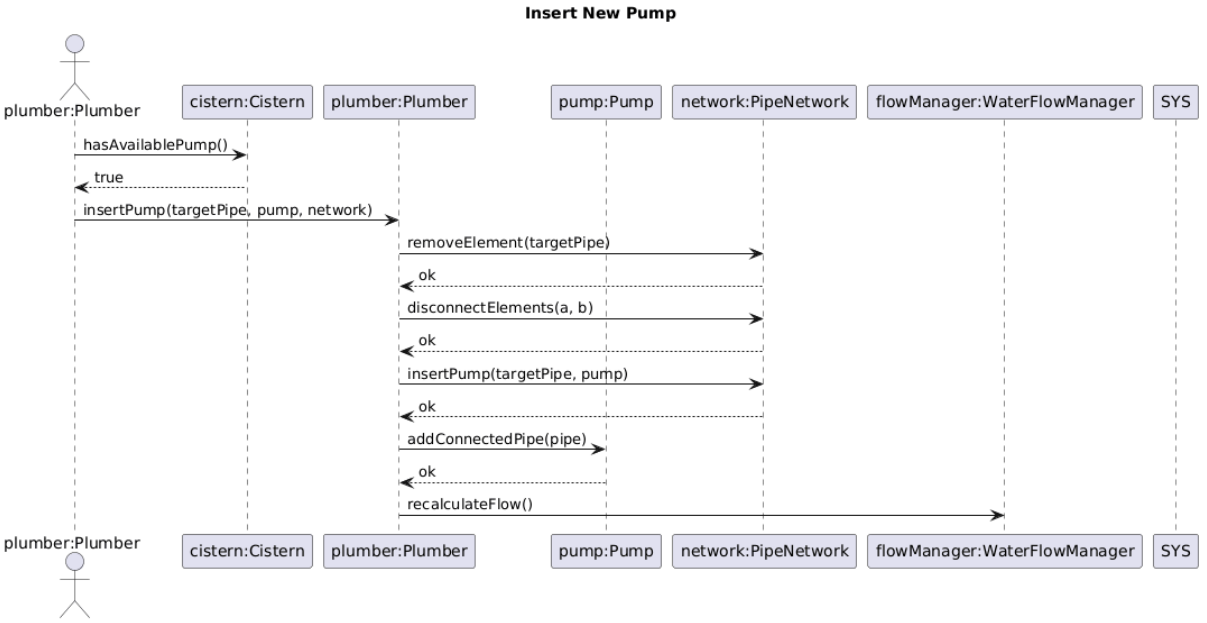
11. **Pick Up Pipe from Cistern**

    A Plumber standing on an adjacent pipe or pump collects a generated pipe from a cistern. The System checks availability at the cistern and marks the pipe as collected. The Plumber can then connect it to the network in a subsequent action.



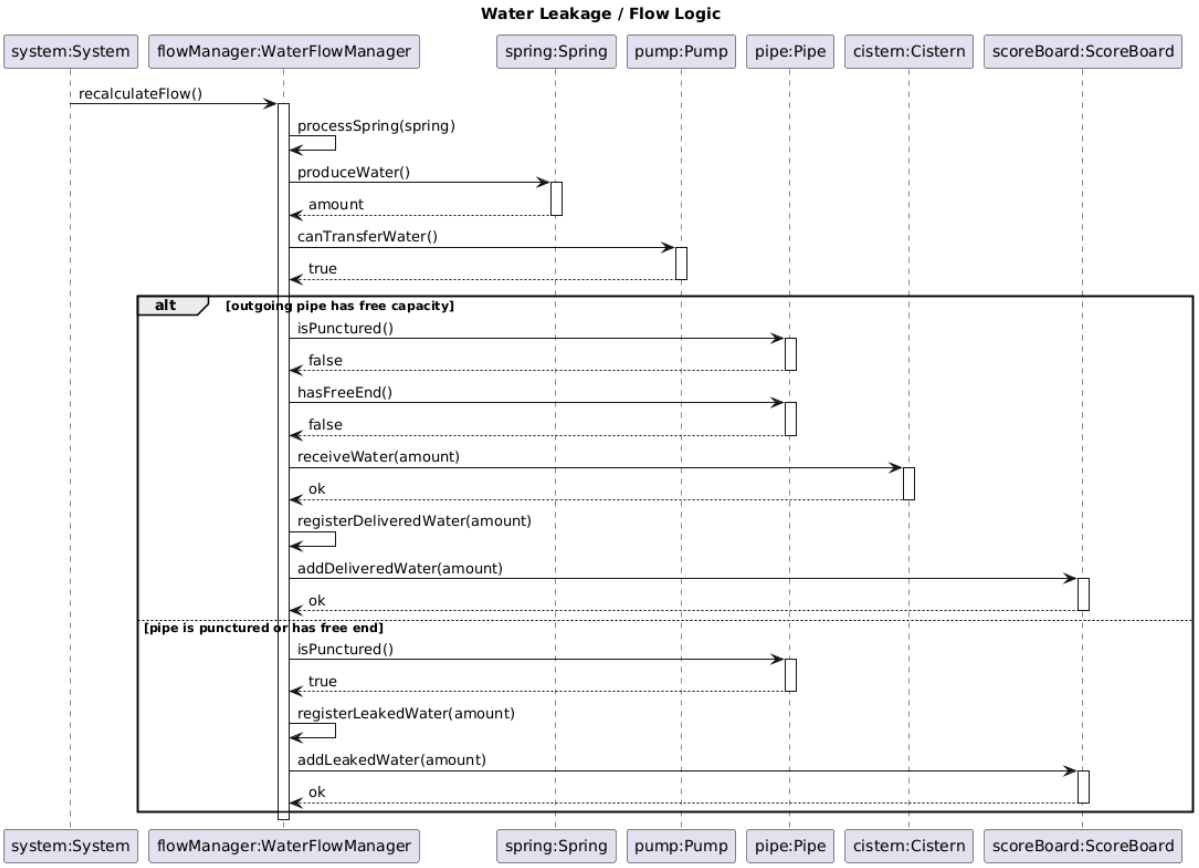
12. **Insert New Pump**

A Plumber who already carries a previously collected pump inserts it into an existing pipe by splitting it into two segments. The network is updated and water flow is recalculated after the insertion.



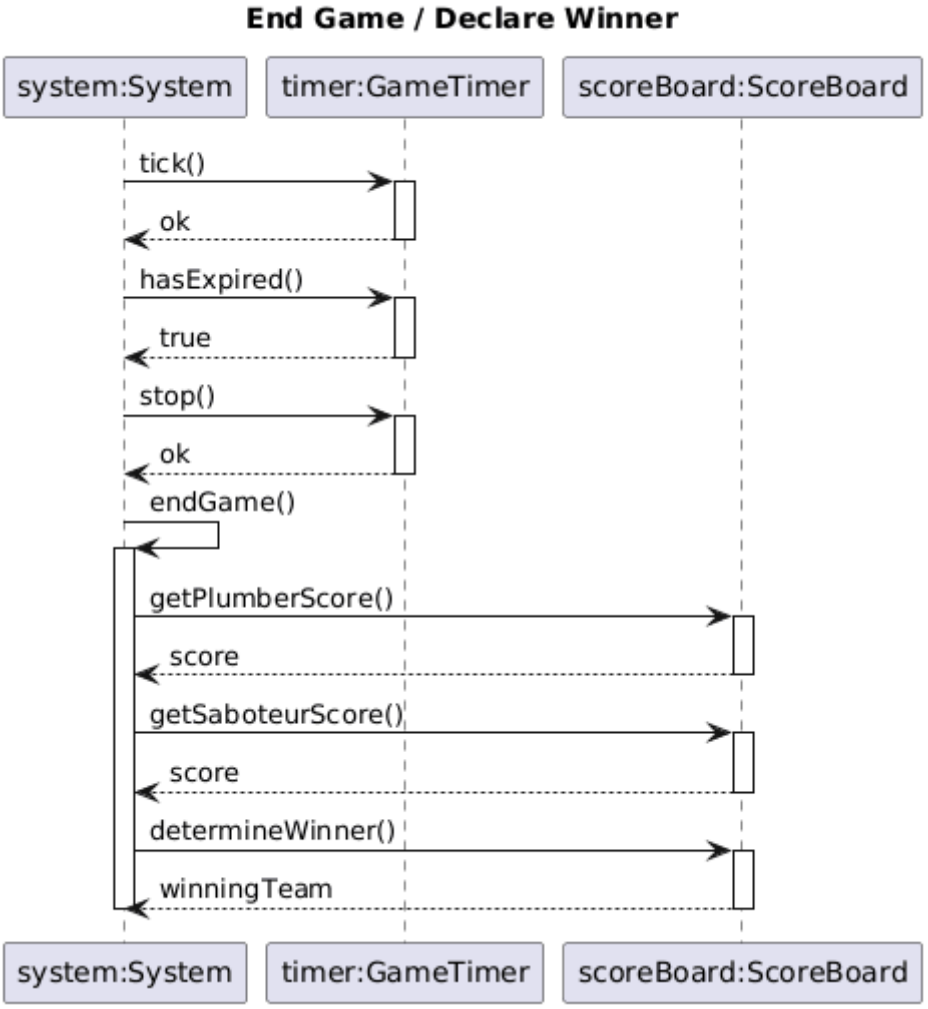
13. **Water Leakage / Flow Logic**

During each game tick the WaterFlowManager processes water movement through the network. A pump transfers its buffered water to the outgoing pipe if capacity allows, then pulls water from the incoming pipe. Water reaching a punctured pipe or a free pipe end leaks to the desert and is counted as lost, while water reaching a cistern is counted as delivered.



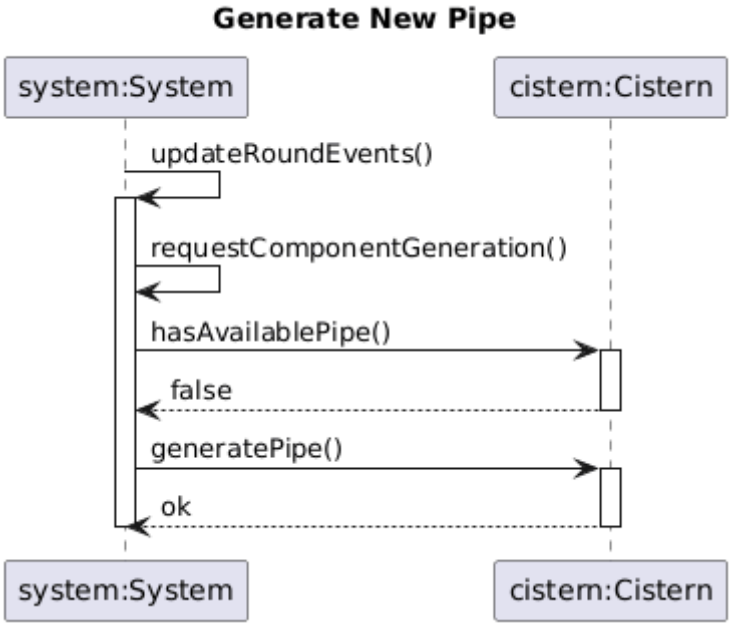
14. **End Game / Declare Winner**

When the game timer expires, the System halts all gameplay and compares total delivered water and total leaked water. If delivered water is greater, the Plumber team wins; if leaked water is greater, the Saboteur team wins; if the two values are equal, the game ends in a draw. Note that the timer-based end condition is a design decision made to resolve the ambiguity in the original problem definition, which only states that the team collecting the most water wins.



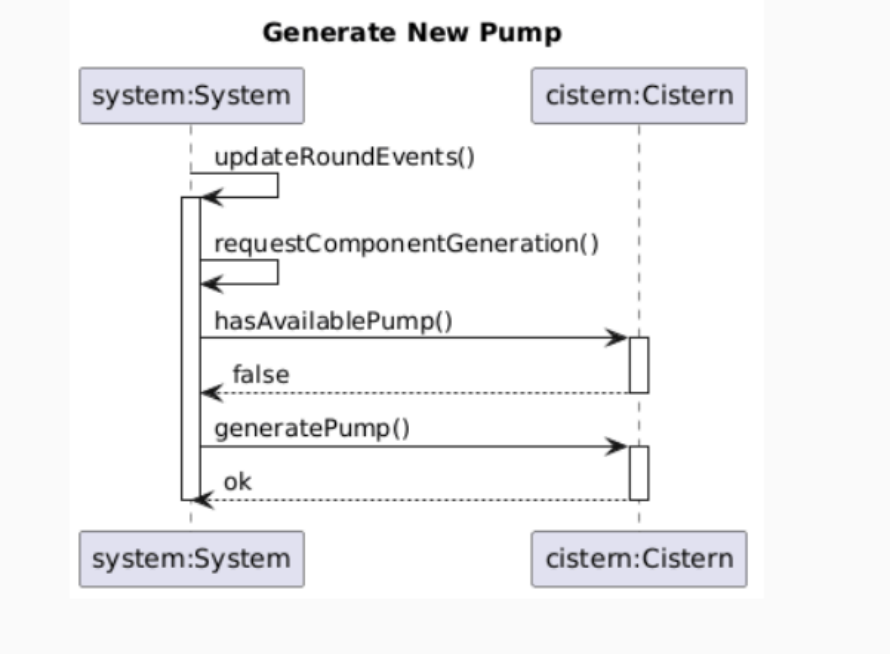
15. **Generate New Pipe**

    The system generates a new pipe at a cistern when the pipe-generation condition is satisfied. The generated pipe remains available at the cistern until collected by a Plumber.



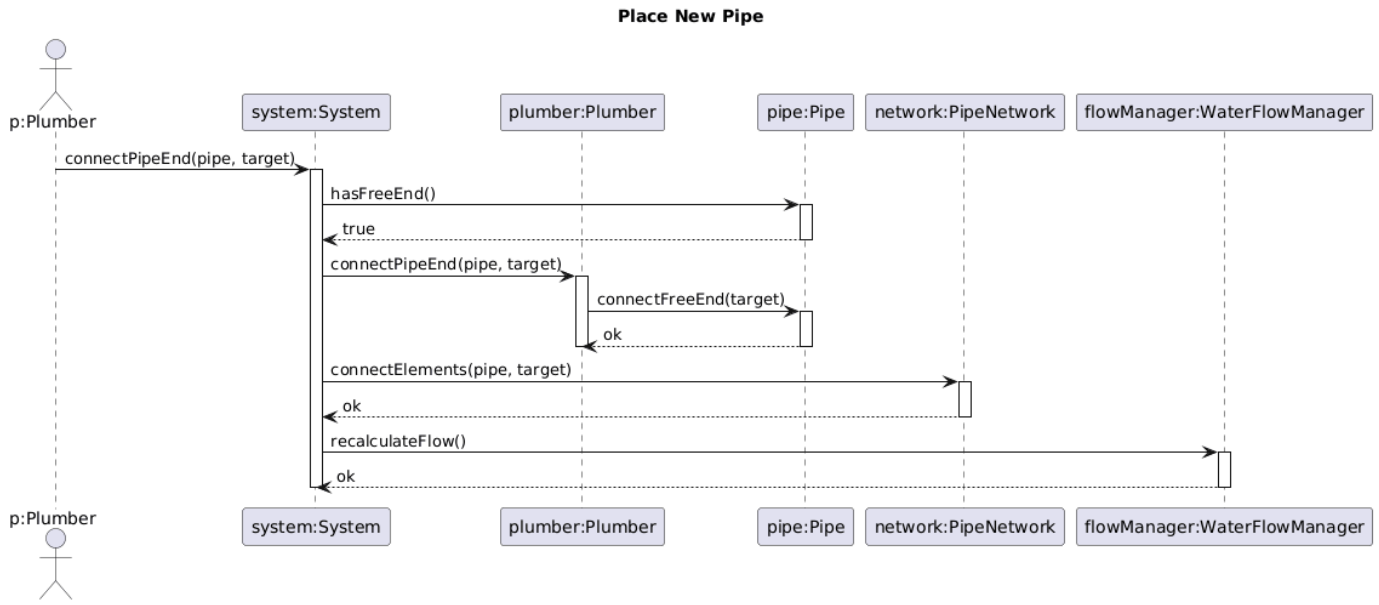
16. **Generate New Pump**

    The system generates a new pump at a cistern when the pump-generation condition is satisfied. The generated pump remains available at the cistern until collected by a Plumber.



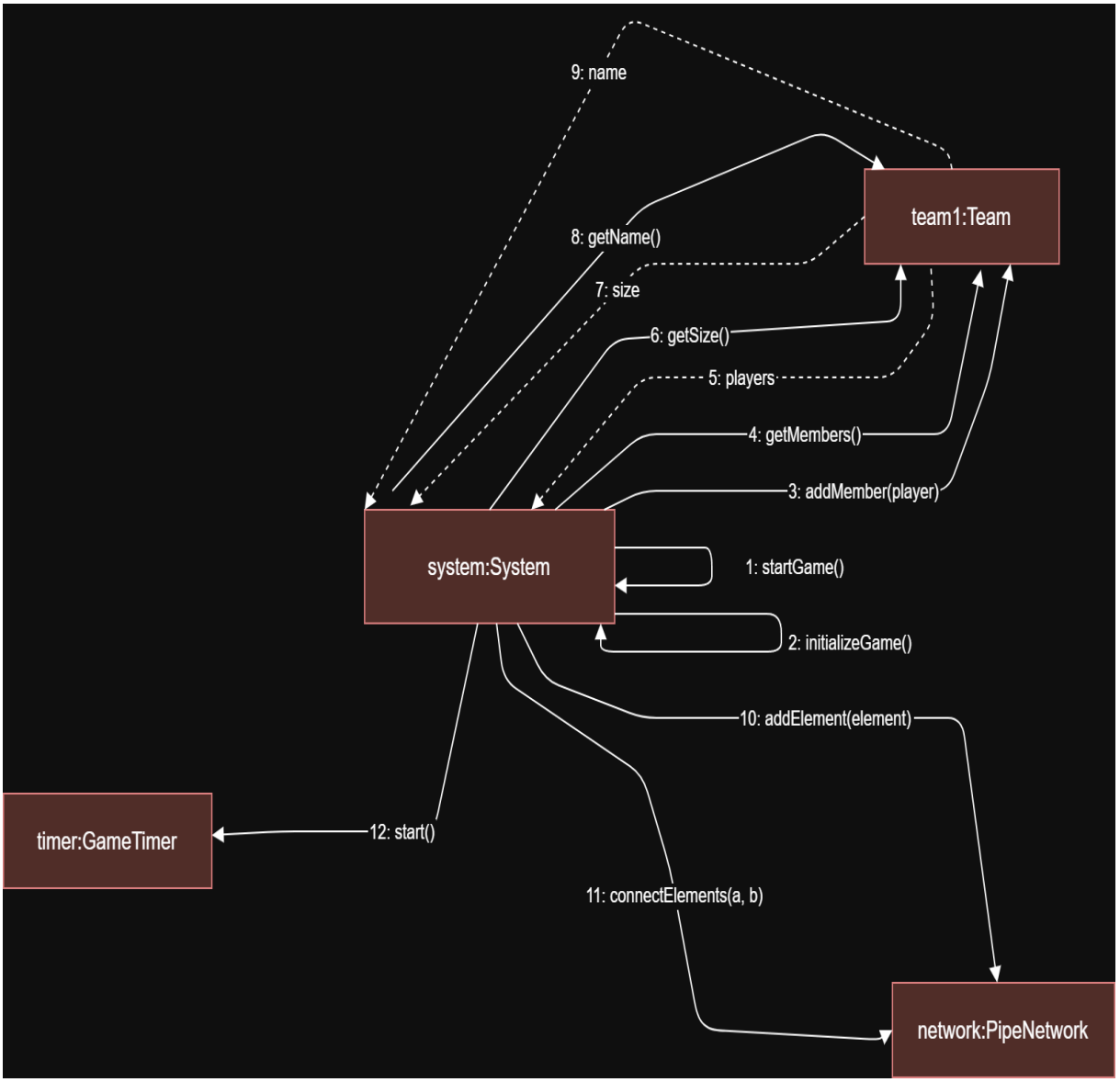
17. **Place New Pipe**

    A Plumber places a previously collected pipe into the network at a valid position. The System validates the placement, inserts the pipe, updates network connections and recalculates water flow.

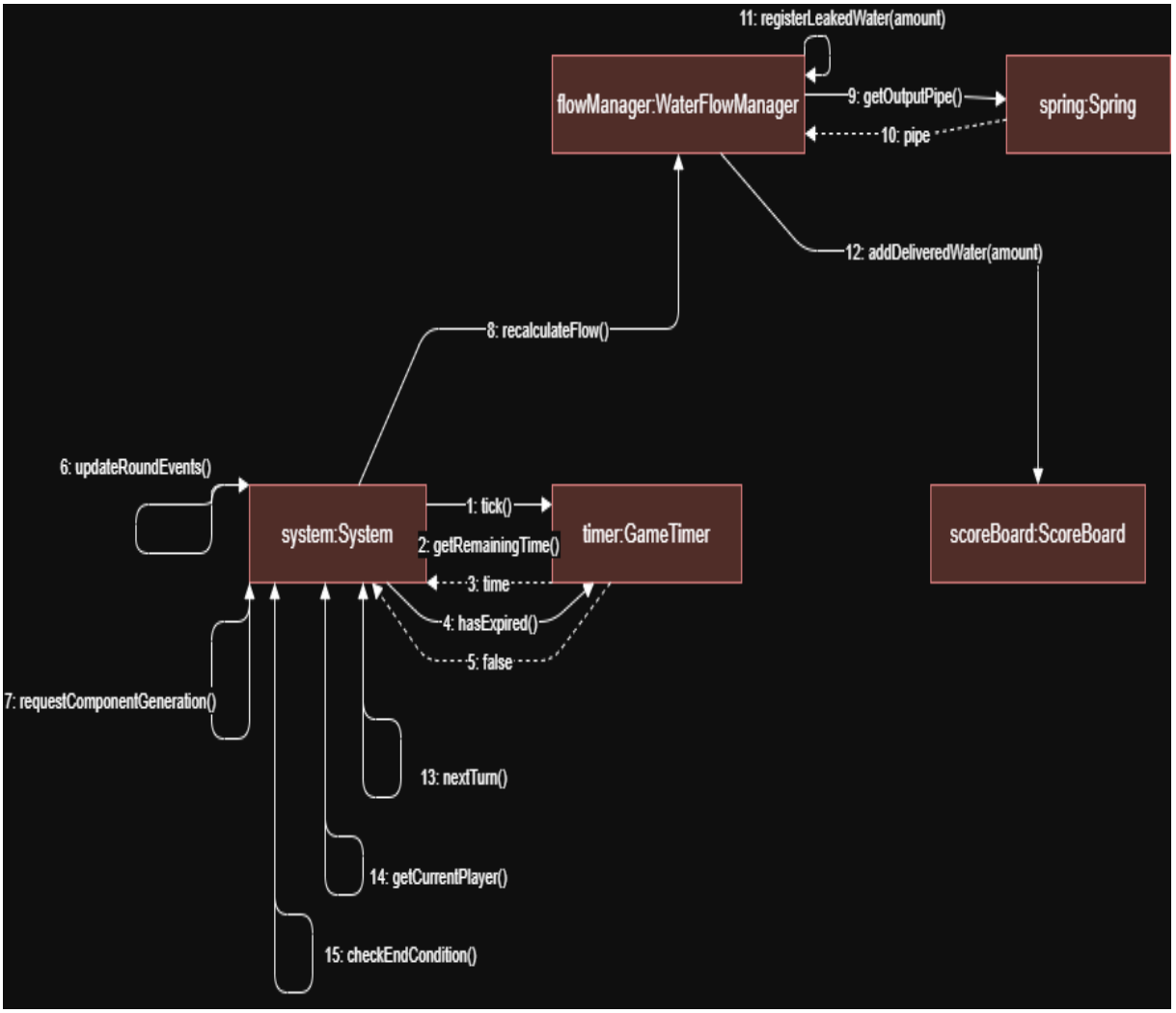

Place New Pipe
p:Plumber
system:System
plumber:Plumber
pipe:Pipe
network:PipeNetwork
flowManager:WaterFlowManager
connectPipeEnd(pipe, target)
hasFreeEnd()
true
connectPipeEnd(pipe, target)
connectFreeEnd(target)
ok
connectElements(pipe, target)
ok
recalculateFlow()
ok
p:Plumber
system:System
plumber:Plumber
pipe:Pipe
network:PipeNetwork
flowManager:WaterFlowManager

5.4 Communication diagrams

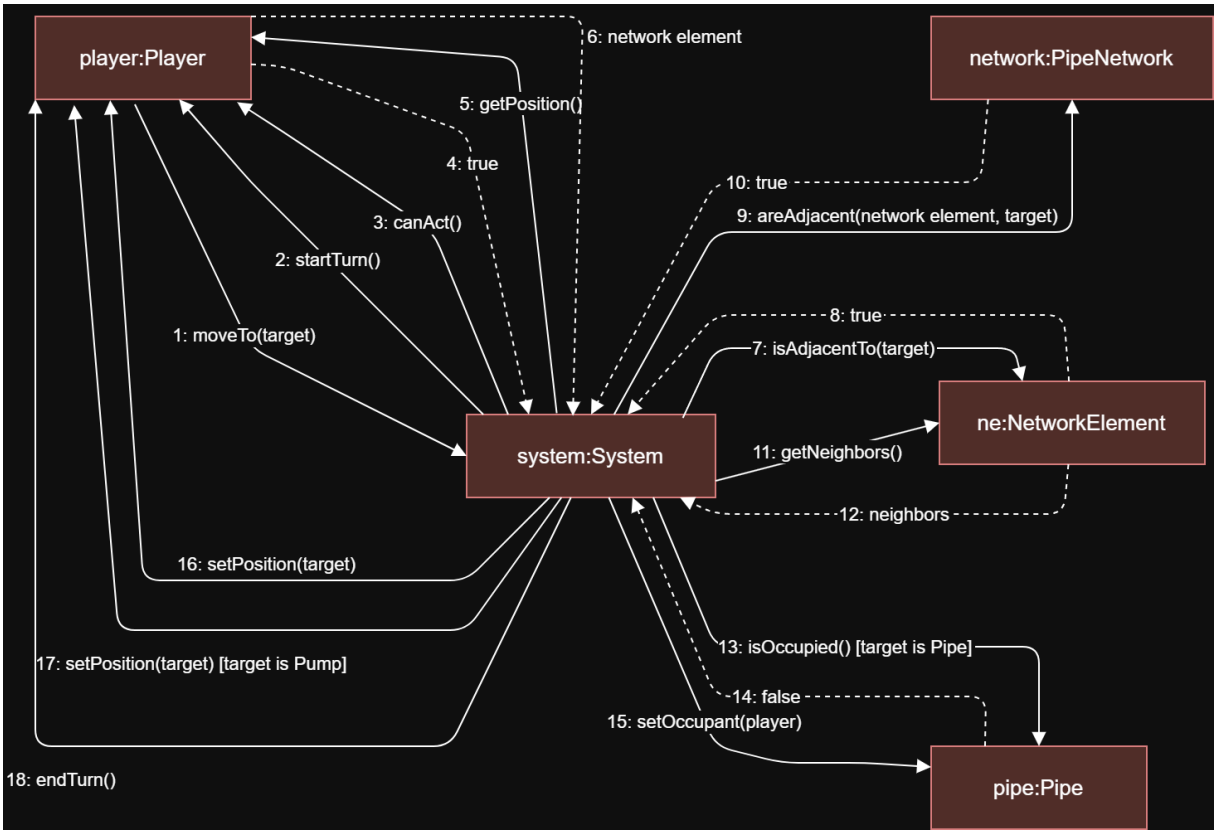
1. Start Game



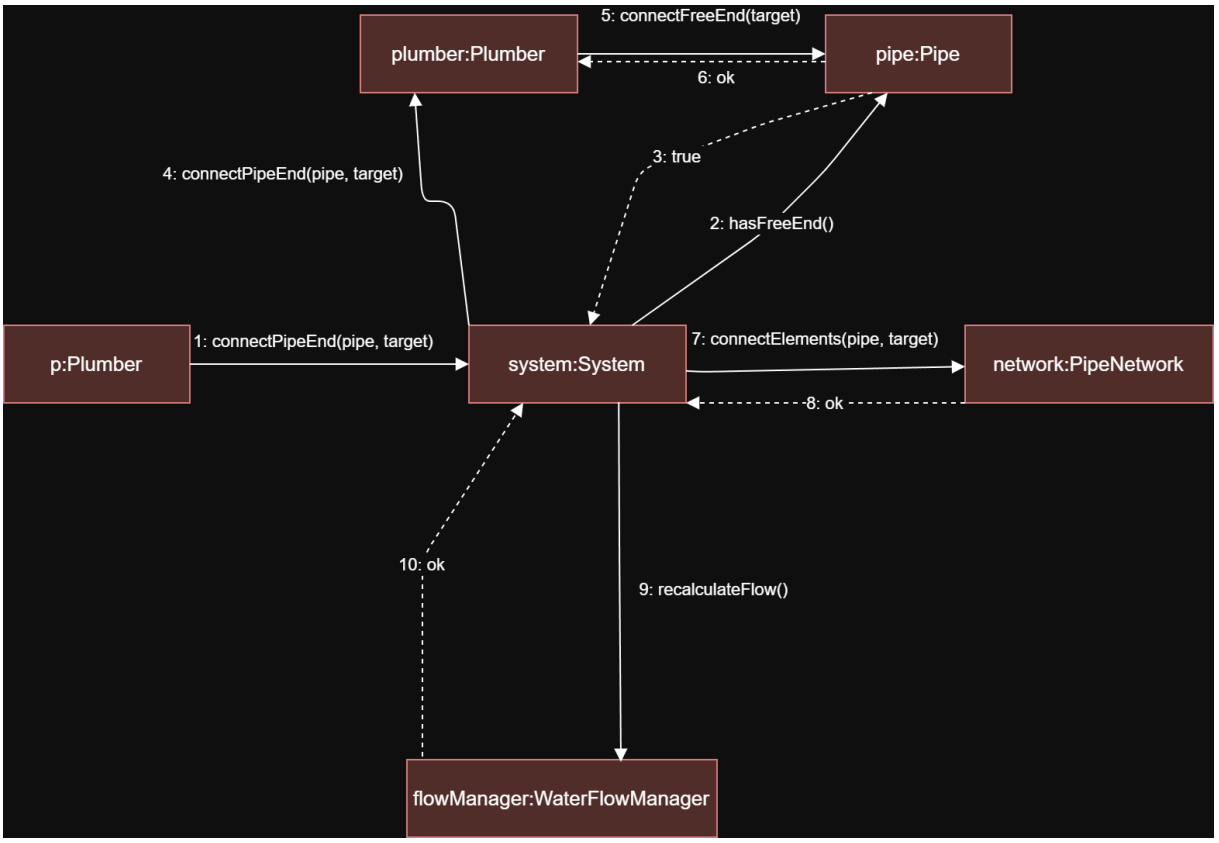
2. Run Game


11: registerLeakedWater(amount)
flowManager:WaterFlowManager
9: getOutputPipe()
spring:Spring
10: pipe
12: addDeliveredWater(amount)
8: recalculateFlow()
6: updateRoundEvents()
system:System
1: tick()
2: getRemainingTime()
timer:GameTimer
scoreBoard:ScoreBoard
3: time
4: hasExpired()
5: false
7: requestComponentGeneration()
13: nextTurn()
14: getCurrentPlayer()
15: checkEndCondition()

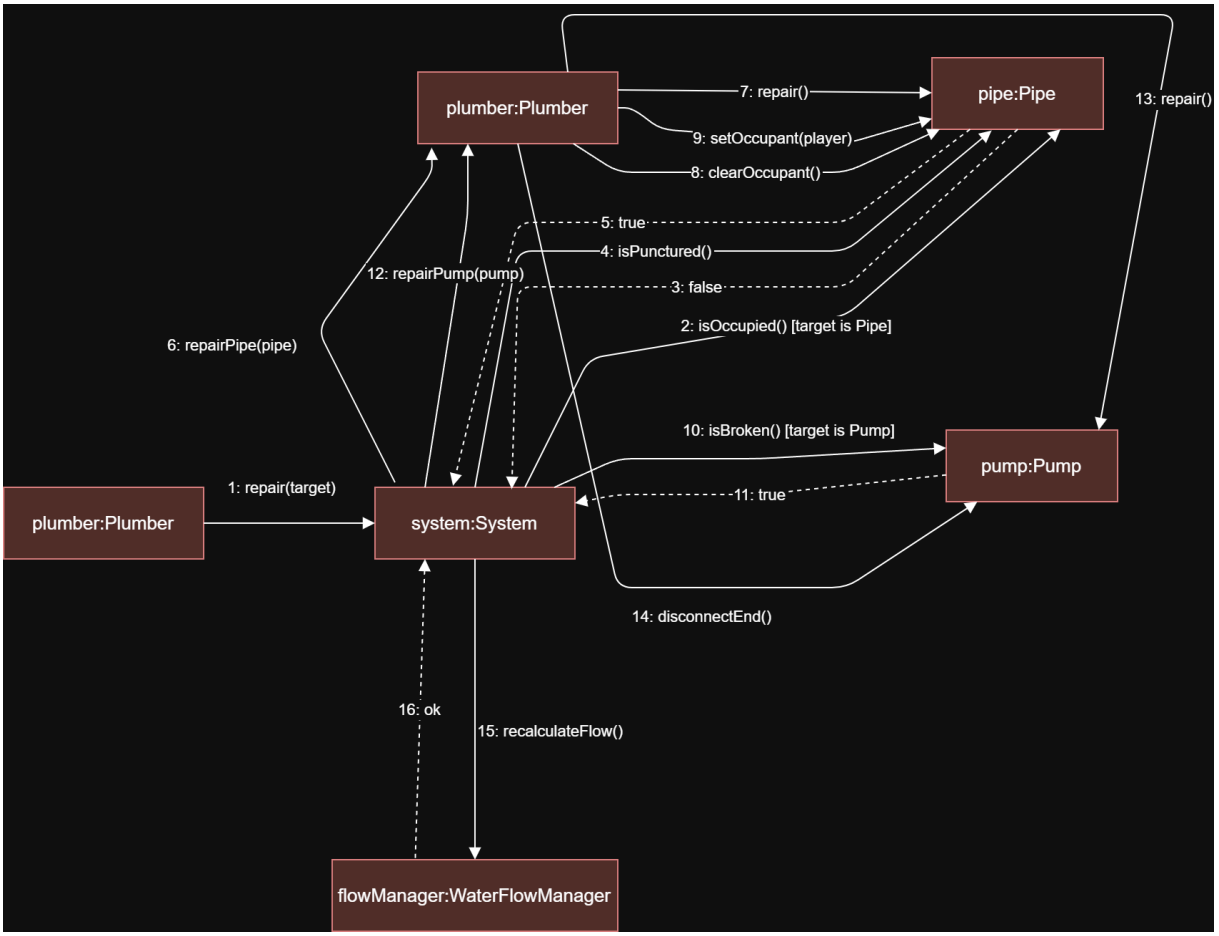
3. Move Player



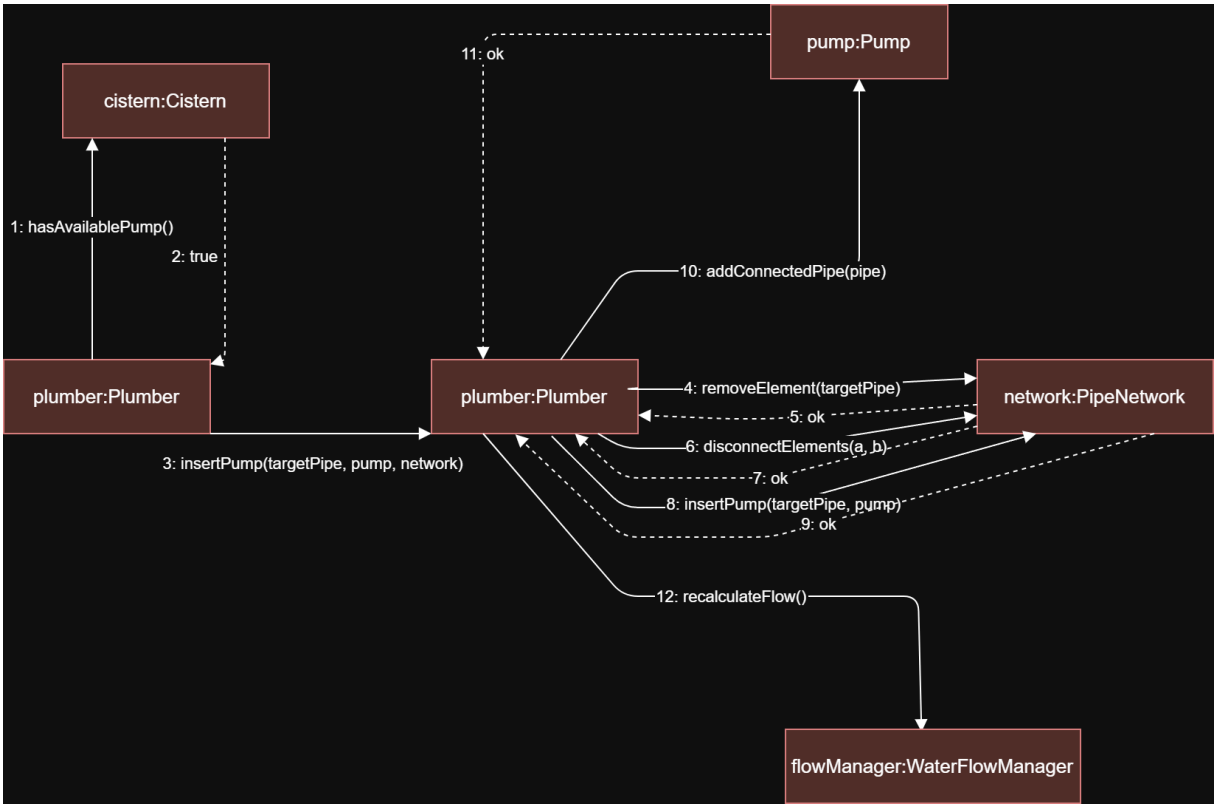
4. Place New Pipe



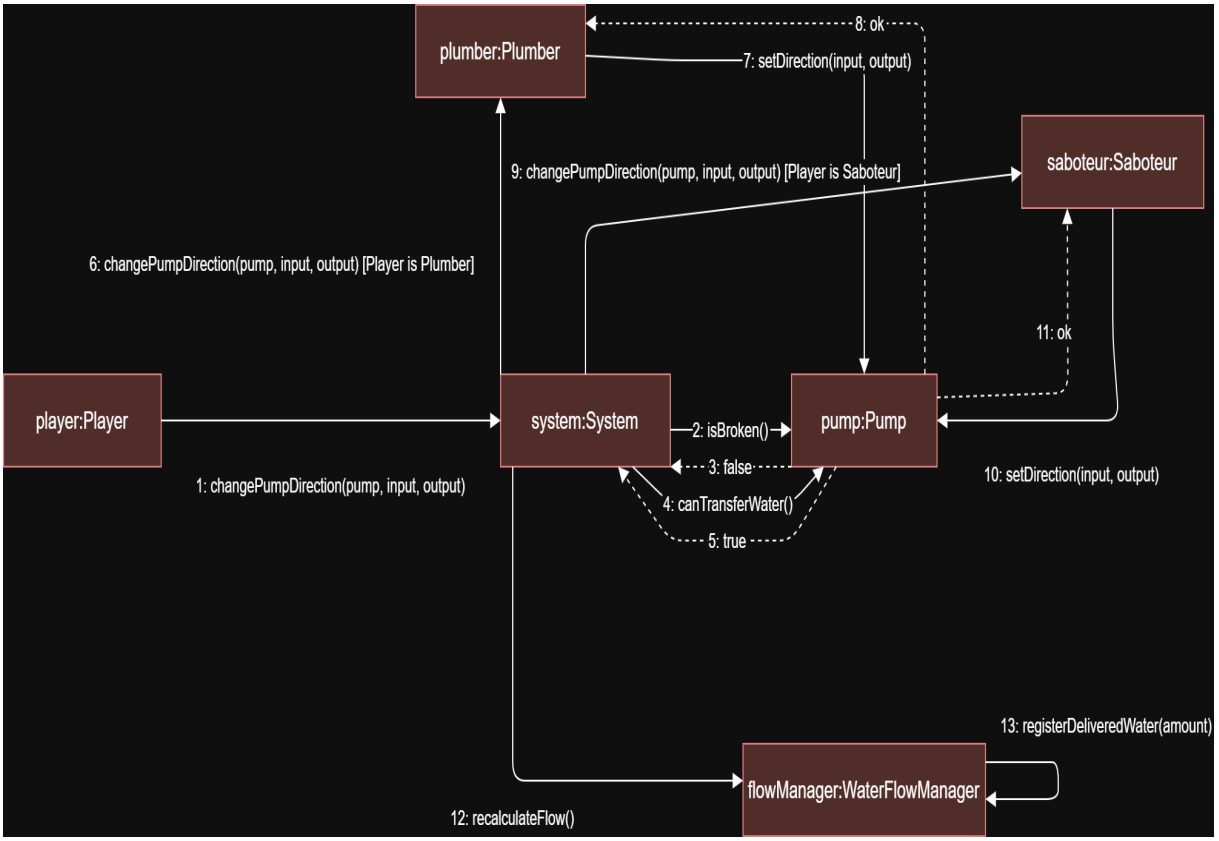
5. Repair Pipe / Pump



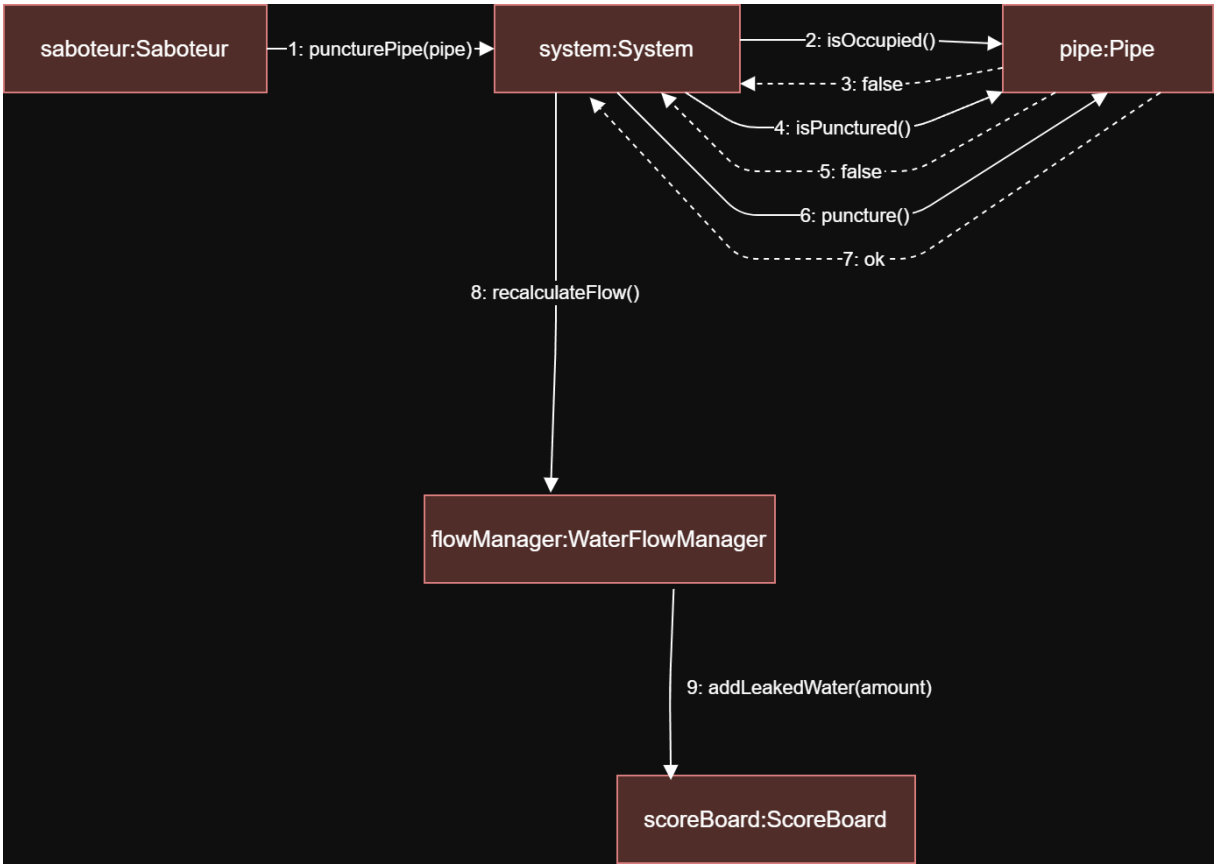
6. Insert New Pump



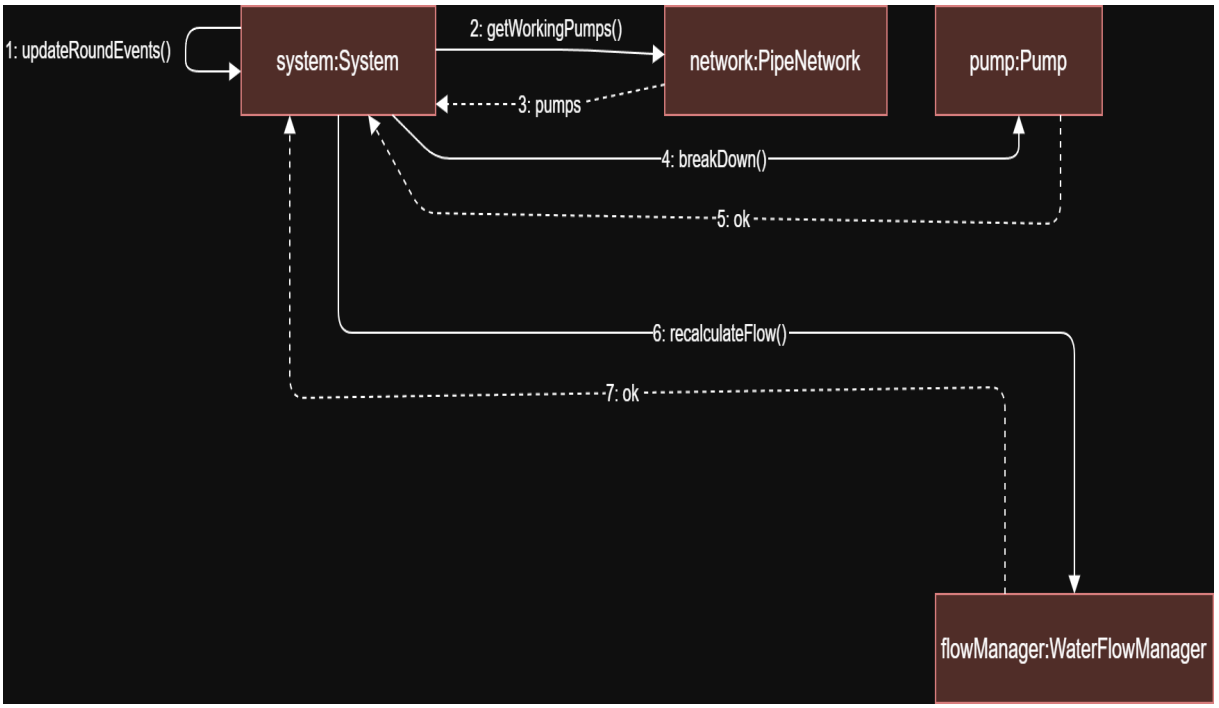
7. Change Pump Direction



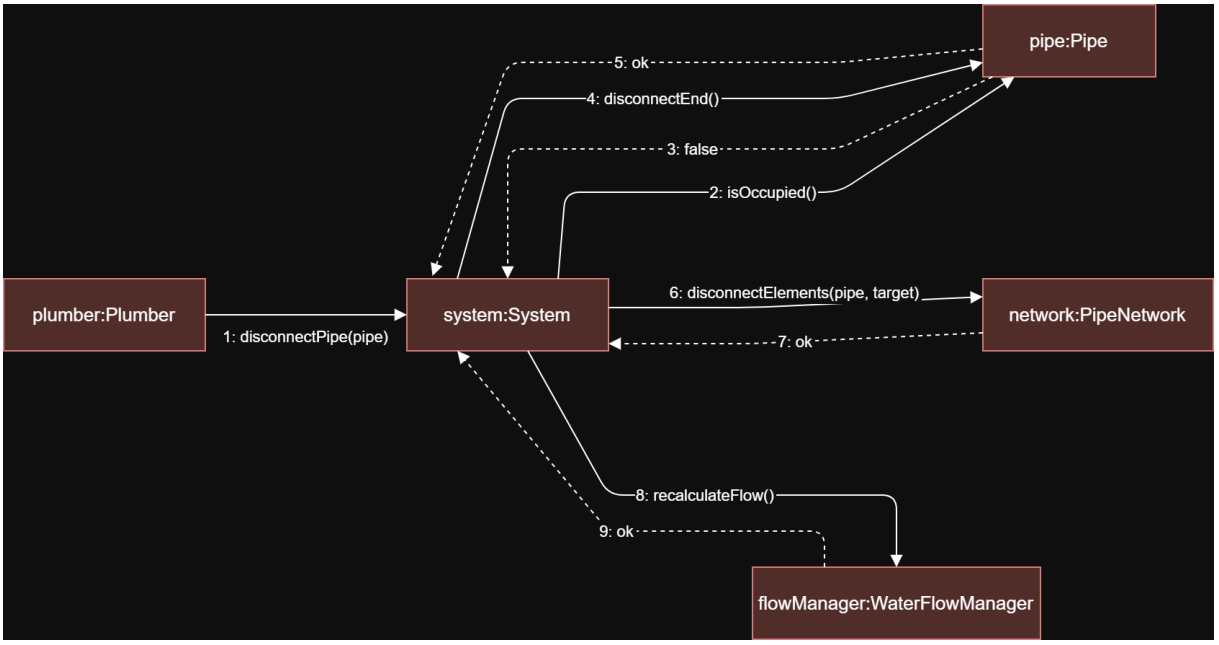
8. Puncture Pipe



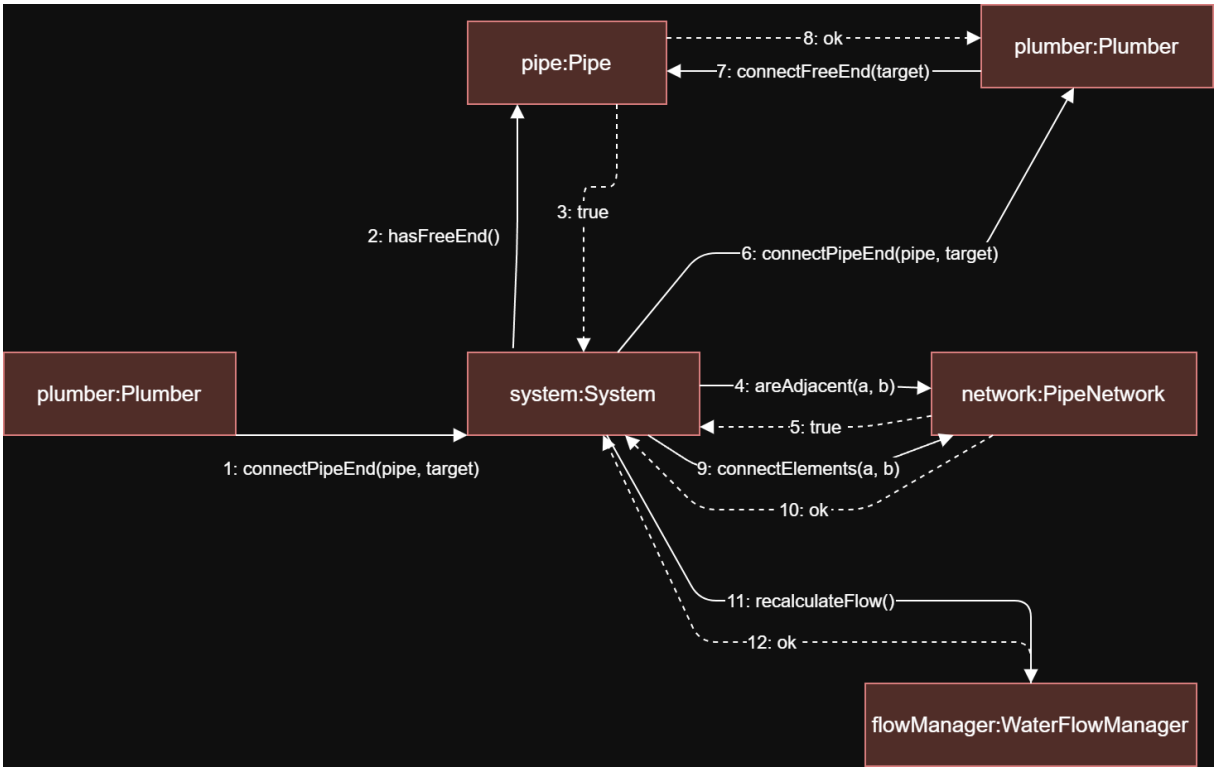
9. Randomly Break Pump



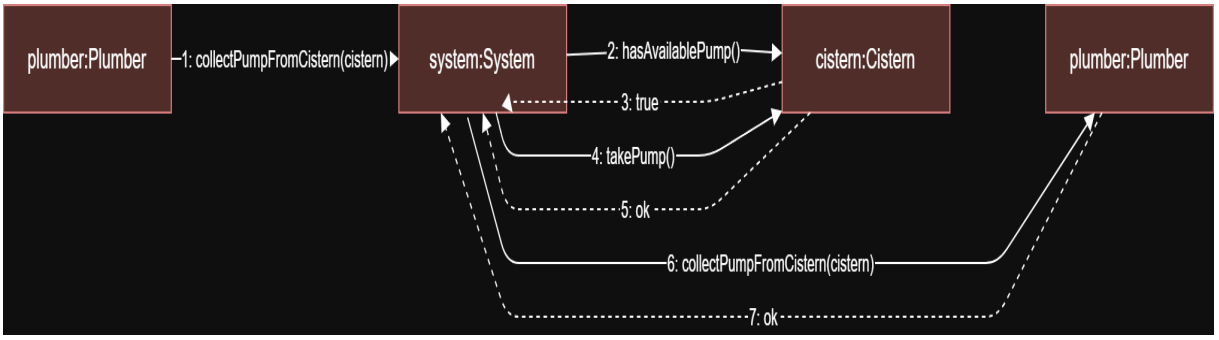
10. Disconnect a Pipe



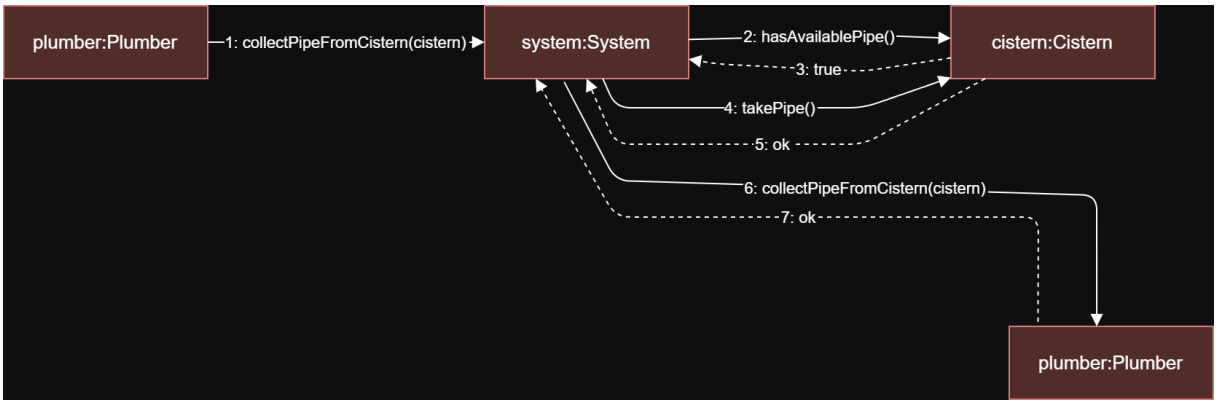
11. Connect a Pipe



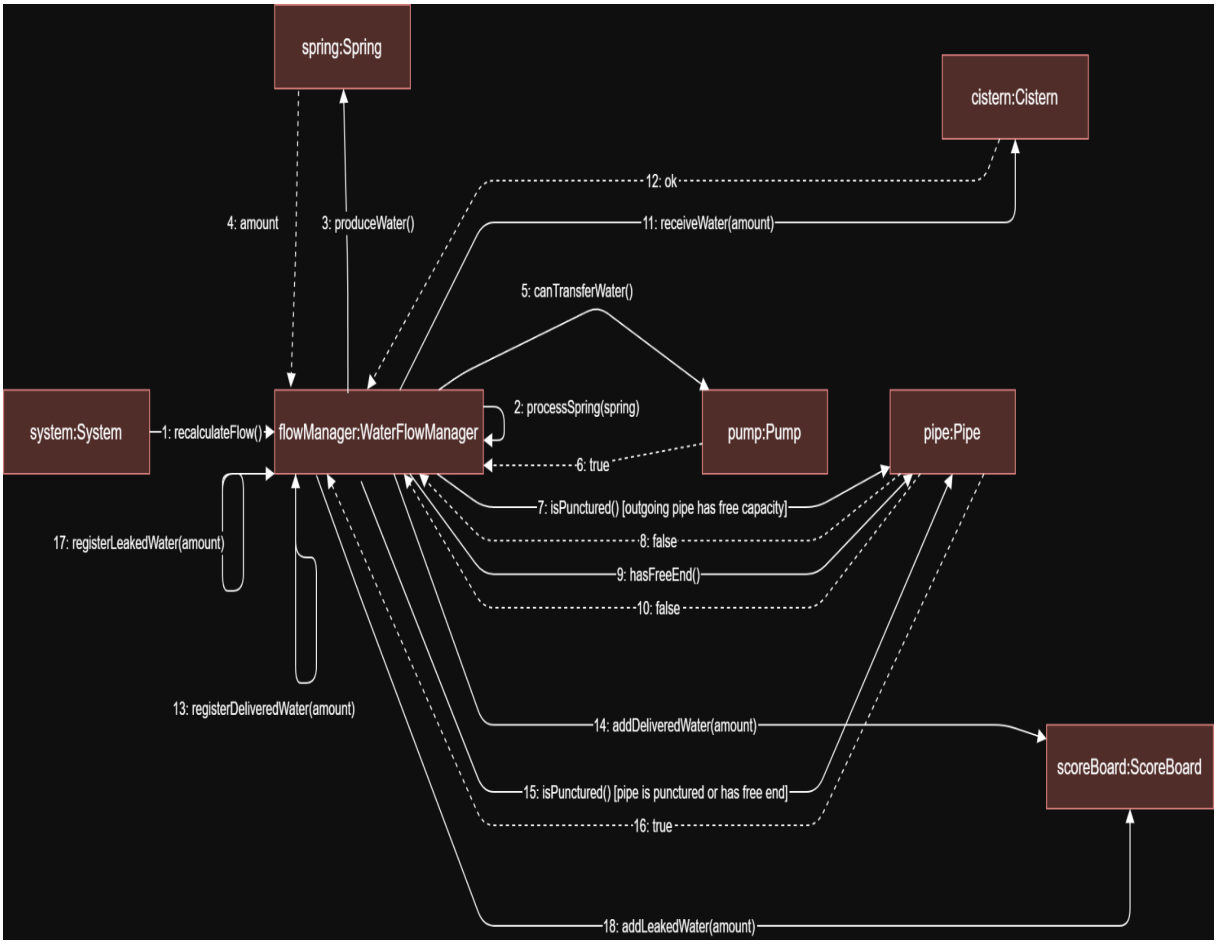
12. Pick Up Pump at Cistern



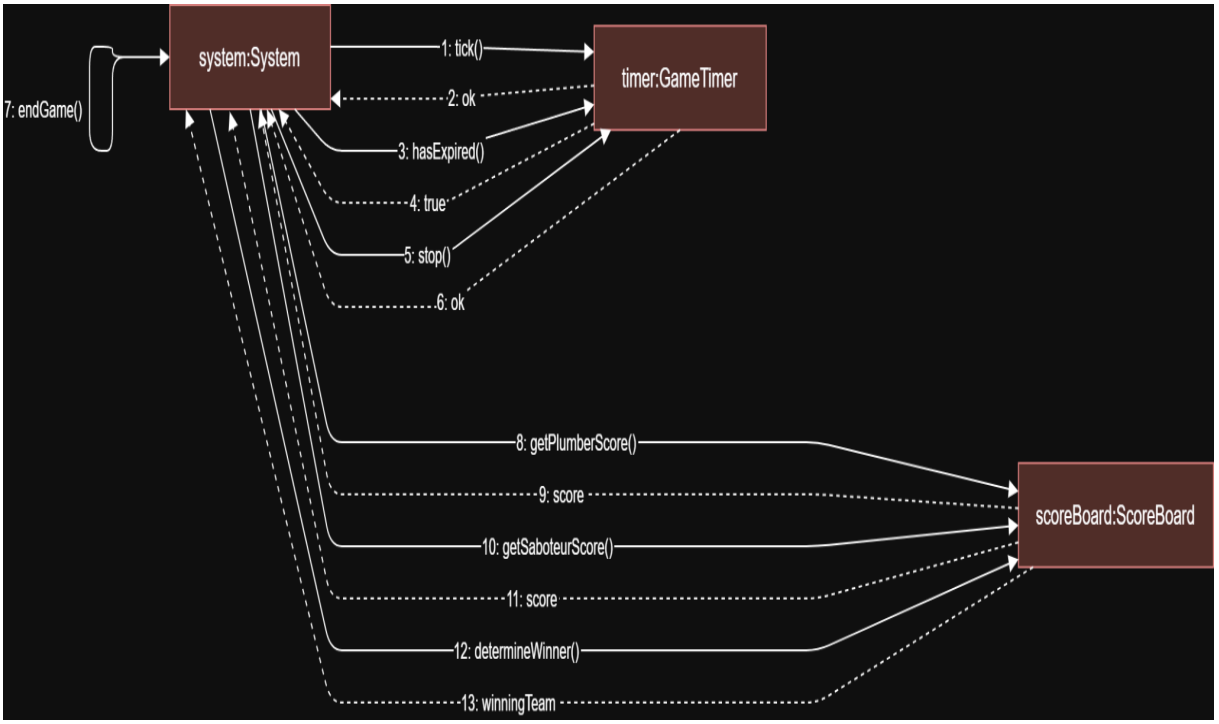
13. Pick Up Pipe from Cistern



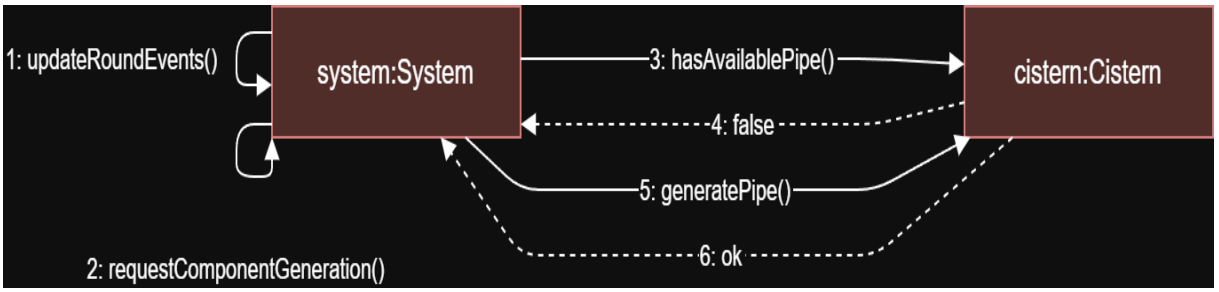
14. Water Leakage / Flow Logic



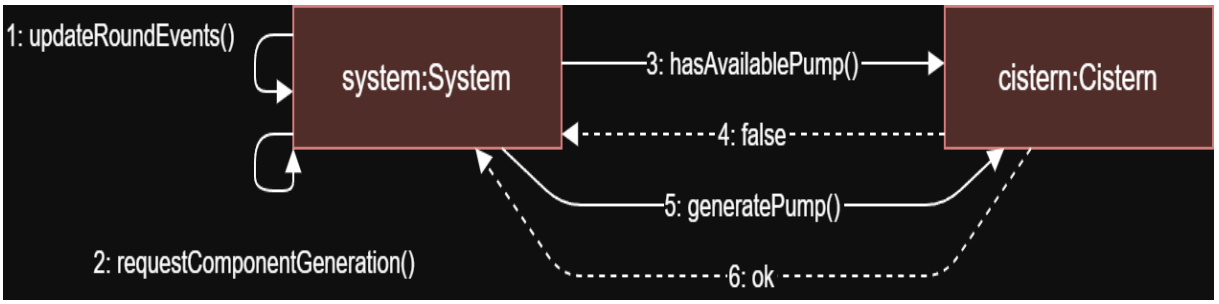
15. End Game / Declare Winner



16. Generate New Pipe



17. Generate New Pump



5.5. Tools Used

Tool	Purpose	Description
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming ideas, and clarifying edge cases in the documentation.
Graphor	UML diagram creation	Used to create UML diagrams such as class, sequence, and activity diagrams.
Draw.io	UML diagram creation	Used by some team members to create communication diagram
PlantUML	UML diagram creation	Used by some team members to generate UML diagrams in text-based format.
Microsoft Word	Documentation	Used to write, format, and organize the analysis model document.
Discord	Communication	Used for online team meetings and coordination between team members.

5. 6. Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
17.03.2026 19:00	2	Ibrahim, Arda, Yahya, Aasif	**Meeting (Discord):** The team discussed the professor's feedback on the Analysis Model (version 2). The goal was to identify required corrections and improvements for the next iteration. The feedback was analyzed in detail, including issues related to sequence diagrams, missing functionalities, model consistency, and traceability. **Decisions:** (1) All missing required sequence diagrams (e.g., puncture pipe, repair, pump direction, etc.) must be added. (2) Inconsistencies between static and dynamic models must be resolved. (3) Responsibilities were delegated in advance so members could work in parallel before the next meeting. (4) The team agreed that Chapter 5.0 (Corrections) must clearly reflect all fixes based on feedback.

21.03.2026 09:00	9	Ibrahim, Arda, Hameez, Ilgın, Yahya, Aasif	Hybrid work session: Ibrahim, Arda, Hameez, and Ilgın worked physically at **Corvinus University library,** while Yahya and Aasif participated remotely. Continuous communication was maintained via WhatsApp. The session began with reviewing and validating all corrections made for Chapter 5.0 to ensure consistency with the professor's feedback. After 12:00, the team proceeded with developing the new skeleton planning documentation. **Decisions and task delegation:** (1) Ilgın: creation of the Use Case Diagram. (2) Arda, Hameez, Ibrahim: writing and aligning Use Case descriptions to ensure consistency with sequence diagrams and functional requirements. (3) Ibrahim: refinement and restructuring of sequence diagrams, including splitting complex diagrams (e.g., Start/Supervise Game) into clearer, smaller diagrams. (4) Aasif: development of "Plans of the skeleton's UI, dialogs". (5) Yahya: creation of Communication Diagrams. (6) Hameez: support in consistency checking across models. The team focused on simplifying complex logic, improving clarity, and ensuring alignment between all model components.
22.03.2026 18:00	4	Ibrahim, Arda, Hameez	**Meeting.** Finalization session. The team performed final consistency checks across the entire document, including use cases, sequence diagrams, UI plans, and communication diagrams. Formatting and structure were revised to match submission requirements. **Decisions:** (1) Minor inconsistencies were corrected to ensure alignment between all diagrams and descriptions. (2) Final protocol and documentation sections were completed. (3) The document was reviewed as a whole to ensure no conflicts remained between static and dynamic models. After validation, the final version was prepared and submitted.